

Du 16/03 au 04/09

Rapport de stage
Système Pick To Light

Enseignant référent : PIGEON ERIC
Tuteur : AL-JRAB Badredine :
Responsable Méthodes et Production



Matys Benoist
Ingénieur 5A Systèmes embarqués 2025-2026



UNIVERSITÉ
CAEN
NORMANDIE

COULIDOR®
Placards et dressings



ESIX NORMANDIE
ÉCOLE SUPÉRIEURE
D'INGÉNIEURS



Table des matières

Remerciements..... 3

Table des abréviations.....4

Table des figures.....5

Introduction..... 6

 Objectifs et durée du stage..... 6

 Problématique..... 7

Présentation de l'entreprise COULIDOOOR.....9

 Présentation de l'entreprise et de ses employés.....9

Présentation de l'environnement de travail..... 11

 Le lieu de travail..... 11

 Les journées de travail..... 13

Présentation des missions.....15

 Présentation générale du système Pick To Light..... 15

 Choix réalisés pour l'architecture du système..... 16

 Fonctionnement du système..... 20

 Alimentation du système..... 24

 Organisation de la pièce.....25

 Tableau de bord Node-RED..... 26

 Difficultés rencontrées et solutions apportées.....27

 Coût total et bénéfices.....29

 Accompagnement du changement.....30

Conclusion.....31

Table des annexes.....32



Remerciements

Avant toute chose, je souhaiterais remercier M. VISSE Arnaud, le président de COULIDOOOR, ainsi que M. FOUCHARD Guillaume, le directeur du site de Verson, de m'avoir intégré dans leur équipe et de m'avoir fait confiance, mais aussi de leur hospitalité, de leur bienveillance et pour le bon déroulement de mon stage au sein de l'entreprise.

J'aimerais également remercier M. AL-JRAB Badredine, le responsable Méthodes et Production de COULIDOOOR mais également mon tuteur de stage, de m'avoir accompagné tout au long du stage, de m'avoir apporté les connaissances théoriques et pratiques concernant l'ERP Silog. Merci également à M. PALLUEL Hervé d'avoir commandé tous les composants nécessaires à la concrétisation de ce projet. De plus, merci à M. GALLANT Damien de m'avoir fourni le matériel nécessaire durant mon stage. Enfin, merci à M. CANTELOUP Victor et M. BRIAND Paul de m'avoir aidé pour la communication de mon système avec Silog qui est l'ERP de l'entreprise.

De plus, je souhaiterais remercier l'école de l'ESIX NORMANDIE et M. PIGEON Eric pour m'avoir permis d'effectuer mon stage au sein de l'entreprise de COULIDOOOR.

Enfin, je suis reconnaissant envers toute l'équipe de COULIDOOOR qui a contribué à ma bonne intégration au sein de l'équipe, dans une ambiance d'entraide et de convivialité où chacun prenait du plaisir à partager son expérience et à m'aider dans la réalisation de mes différentes missions.

Merci à tous pour le bon déroulement de ce stage qui m'aura permis de mieux comprendre le rôle et l'importance de chacun au sein d'une entreprise et au sein d'un projet.



Table des abréviations

SAV : Service après-vente

PTL : Pick To Light : système allumant un voyant lumineux pour orienter l'opérateur vers une référence qu'il doit prélever dans un bac

ERP : Enterprise Resource Planning : logiciel de gestion que les entreprises utilisent pour gérer leurs activités quotidiennes comme par exemple la comptabilité, les achats, la gestion de production, la gestion des non-conformités, ainsi que les opérations de supply chain

SAS : Société par actions simplifiée

CFR : Couldoor France Rangement

CNO : Couldoor Nord-Ouest

Table des figures

Figure 1 : Diagramme d'ishikawa retraçant les sources potentielles d'erreurs.....	7
Figure 2 : Carte des sites de COULIDOOOR en France.....	9
Figure 3 : Organigramme de l'équipe COULIDOOOR à Verson.....	10
Figure 4 : Image du site de COULIDOOOR à CFR.....	11
Figure 5 : Organigramme présentant la chaîne de production de Coulidoor.....	12
Figure 6 : Image des EPI nécessaires sur site.....	13
Figure 7 : Images des logos des logiciels utilisés.....	13
Figure 8 : Schéma fonctionnel du système Pick To Light.....	15
Figure 9 : Image de la Raspberry Pi 5.....	16
Figure 10 : Image d'un ESP32.....	17
Figure 11 : Image d'un bandeau de leds ws2812.....	17
Figure 12 : Image de connecteurs JST-SM.....	18
Figure 13 : Image d'un bandeau de leds.....	18
Figure 14 : Image d'un connecteur Wago et d'un USB-c breakout.....	19
Figure 15 : Image des leds avec les profilés et les caches opaques.....	19
Figure 16 : Organigramme du fonctionnement du Raspberry pi 5.....	20
Figure 17 : Organigramme du fonctionnement des ESP32.....	22
Figure 18 : Schéma représentatif du câblage du système.....	24
Figure 19 : Schéma représentatif de la pièce.....	25
Figure 20 : Image de la pièce avec le système en fonctionnement.....	25
Figure 21 : Image du tableau de bord Node-RED.....	26
Figure 22 : Image d'un bandeau de leds avec un cache thermique.....	28
Figure 23 : Image du boîtier 3D avec les composants alimentation à l'intérieur.....	29
Figure 24 : Tableur des coûts du système Pick To Light.....	29

Introduction

Objectifs et durée du stage

Dans le cadre de ma troisième et dernière année en école d'ingénieurs en Mécatronique et Systèmes Embarqués, option Systèmes Nomades à l'ESIX NORMANDIE à Caen, j'ai réalisé mon stage au sein des locaux de l'entreprise COULIDOOOR à Verson du lundi 16 mars au vendredi 4 septembre 2026.

J'ai choisi d'effectuer mon stage à COULIDOOOR car le projet de créer un Système Pick To Light m'attirait, d'autant plus que j'avais une idée de comment relier les connaissances sur les systèmes embarqués, la mécanique, la programmation et l'électronique que j'ai acquises durant mes 3 années d'études dans l'école de l'ESIX NORMANDIE aux connaissances concernant l'internet des objets et la programmation que j'ai acquises au sein de l'École de Technologie Supérieure de Montréal durant ma mobilité de 4 mois au Canada. J'étais curieux et enthousiaste à l'idée de savoir comment je pouvais réaliser ce projet en totalité en utilisant toutes les connaissances que j'ai acquises durant mon cursus scolaire.

Lors de ce stage, j'ai eu l'opportunité de concevoir, de réaliser, de tester et d'implanter un système Pick To Light pour le service de la préparation d'accessoires. J'ai ainsi conçu un système utilisant des microcontrôleurs, des notions d'électricité et de communication numérique. Ce projet m'a permis d'aborder les aspects techniques (schématisation du système, tests, optimisation), organisationnels (collaboration avec différentes équipes, suivi de l'évolution du projet et comparaison avec la planification) et méthodologiques (présentations du système à l'entreprise pour validation et modifications suites à ces présentations) dans un projet industriel.

Les objectifs de ce stage étaient pour l'entreprise d'avoir un système permettant aux opérateurs d'avoir un nouveau détrompeur visuel pour les aider lors de la préparation de commandes d'accessoires. Celui-ci avait pour but de diminuer le nombre de SAV provenant du service de préparation d'accessoires qui représentent 42 % des SAV de l'entreprise liés à la production.

Mes objectifs durant ce stage étaient de mettre à profit mes connaissances afin de réaliser le système demandé par l'entreprise pour qu'elle puisse le mettre à profit. Je devais également trouver une solution modulable qui permette l'ajout d'éléments et facilitant la maintenance, c'est-à-dire que la dysfonction d'un élément peut être résolue de manière locale sans impacter tout le système. Je devais également accompagner l'entreprise avec les changements apportés par mon système pour qu'ils puissent le modifier ou le réparer en cas de panne.

Problématique

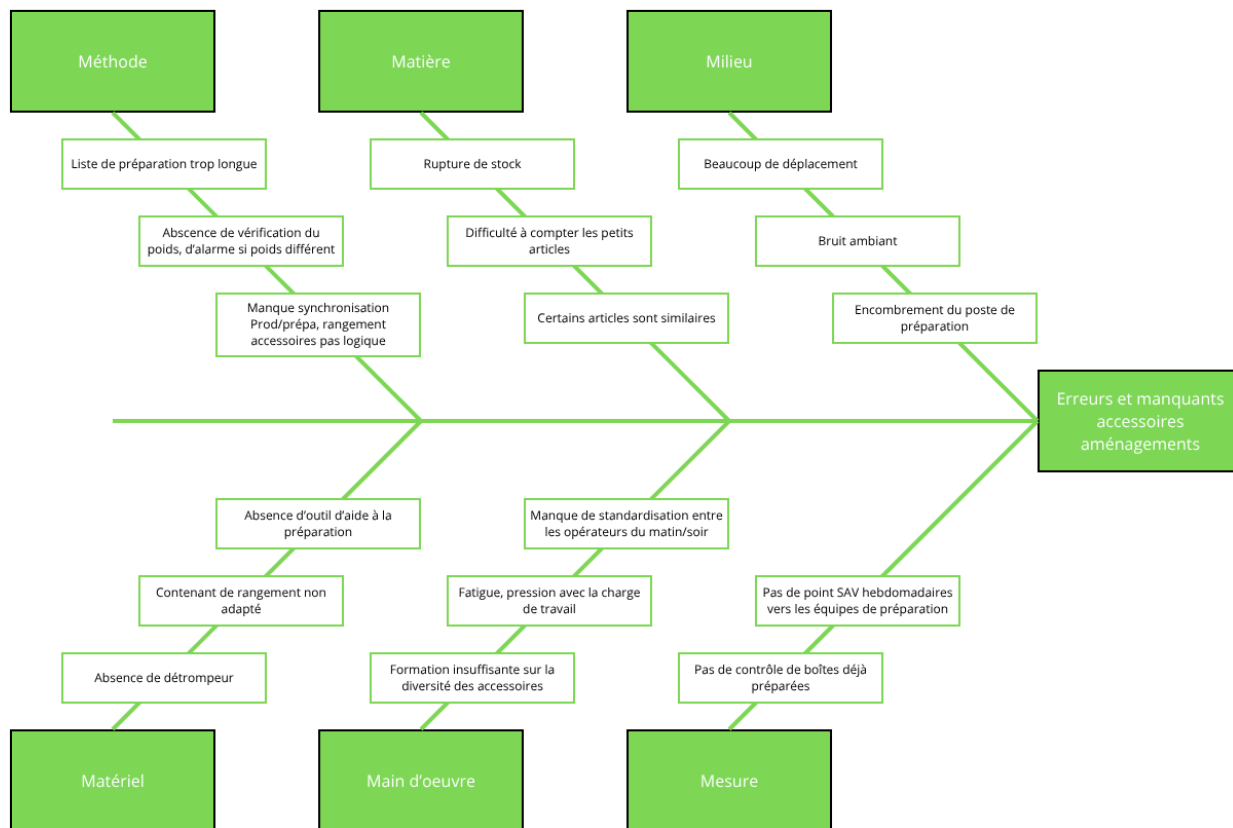


Figure 1 : Diagramme d'ishikawa retraçant les sources potentielles d'erreurs

Le service de la préparation d'accessoires est soumis à une gestion 100 % humaine, ce qui peut amener des erreurs lors de la préparation de commande. Ceci peut être dû aux différentes sources présentées sur le diagramme d'ishikawa présent en figure 1. Ces erreurs sont la source de 42 % des SAV de l'entreprise provenant de la production. C'est dans un contexte de tentative de résolution des principales sources d'erreurs identifiées que j'ai intégré l'entreprise.

Le diagramme d'Ishikawa ou diagramme des 6M a permis d'identifier les sources potentielles des erreurs commises. Tout d'abord, concernant les méthodes, l'entreprise a identifié que les erreurs pouvaient venir des listes de préparation trop longues, qu'il n'y a pas une vérification systématique entre le poids théorique et le poids réel, d'autant plus que le poids théorique n'est pas toujours fiable, de plus il n'y a pas d'alerte en cas de différence de poids. Ensuite, un manque de synchronisation entre le flux de production et de préparation a été constaté de même que la rupture d'accessoires n'est pas formalisée et la procédure de rangement d'accessoires n'est pas définie de façon logique.

Concernant la matière donc les articles que les opérateurs doivent prélever. L'entreprise a constaté des ruptures fréquentes de stock, une difficulté à compter les petites pièces, une difficulté à différencier des articles similaires et des articles à recouper qui peuvent présenter des erreurs de cotation.

Benoist Matys, Stage en IoT
2025-2026

En ce qui concerne le milieu, il a été observé que les opérateurs ont beaucoup de déplacements à effectuer, le milieu dans lequel ils travaillent est bruyant avec les machines en activité et leur poste de travail est souvent encombré ce qui peut nuire à leur concentration.

Pour ce qui est des outils mis à leur disposition, l'entreprise a identifié un manque de détrompeurs, des contenants non adaptés pour ranger les articles et une absence d'outils d'aide à la préparation.

Au sujet des opérateurs, il a été mis en évidence que les opérateurs manquent de formation concernant la diversité des articles, ceux-ci sont sujets à de la fatigue à cause de la charge de travail et de la pression ressenties en cas de préparation d'urgence. Enfin il est apparu qu'il y a un manque de standardisation entre les opérateurs du matin et du soir.

Au sujet des contrôles, il s'est avéré que les boîtes déjà préparées ne sont pas contrôlées et les points concernant les SAV ne sont pas faits de manière régulière auprès des équipes de préparation.

C'est donc au niveau du matériel que mon système s'intègre. Celui-ci a pour but d'ajouter un détrompeur visuel à l'équipe de préparation, leur ajoutant ainsi un outil d'aide à la préparation.

Le système Pick To Light que je devais concevoir, réaliser et implanter a pour but de résoudre ou du moins limiter les erreurs humaines survenant lors de la préparation des commandes. Le système agit donc comme un détrompeur permettant de limiter les erreurs possibles. Il a été vu avec l'entreprise que le principal détrompeur utilisé dans ce système sera visuel avec l'allumage de leds colorées permettant d'indiquer aux opérateurs les articles à prélever.

Le système Pick To Light que j'ai implanté intègre des microcontrôleurs (Raspberry Pi 5 et ESP32), des actionneurs (leds adressables WS2812), des alimentations, des USB-C breakouts et des connecteurs (JST-SM et wago). Mais le système ne doit pas seulement allumer des leds, il doit communiquer avec l'ERP Silog pour savoir quelles leds il est nécessaire d'allumer. Il doit également être modifiable et permettre l'ajout de nouvelles références. Enfin, la maintenance doit être simple, une panne doit être locale et ne doit pas impacter tout le système, la réparation doit être locale et rapide. Enfin, pour pérenniser le système, il est nécessaire d'expliquer à l'entreprise les actions à réaliser pour ajouter des références ou pour régler une panne.

Ce rapport a pour but de répondre à la problématique suivante : "Comment concevoir, réaliser et implanter un système Pick To Light qui soit efficace, modifiable et permettant une maintenance rapide et accessible dans un cadre stricte avec des contraintes ?"

Dans ce rapport, je présenterai le contexte organisationnel puis technique de ce projet, puis je détaillerai les différents travaux réalisés durant mon stage avec les problèmes

Benoist Matys, Stage en IoT
2025-2026

rencontrés, les choix effectués et les solutions apportées avant d'analyser les enseignements tirés de cette expérience et les compétences développées.

Présentation de l'entreprise COULIDOOOR

Présentation de l'entreprise et de ses employés

COULIDOOOR est une SAS spécialisée dans la fabrication de meubles de l'habitat particulièrement le mobilier dédié à la chambre à coucher et au dressing, à la salle à manger, au salon de séjour, les meubles de séparation de pièces, les verrières, le mobilier de cuisine, de salle de bain, de bureau. De manière générale, toute activité de menuiserie industrielle, la commercialisation et la distribution de mobilier avec un atout majeur : la fabrication 100% sur mesure. Elle a été fondée en 1986 et elle est présidée par M. VISSE Arnaud. Elle possède un capital social de 1 500 000 € et compte plus de 270 collaborateurs.

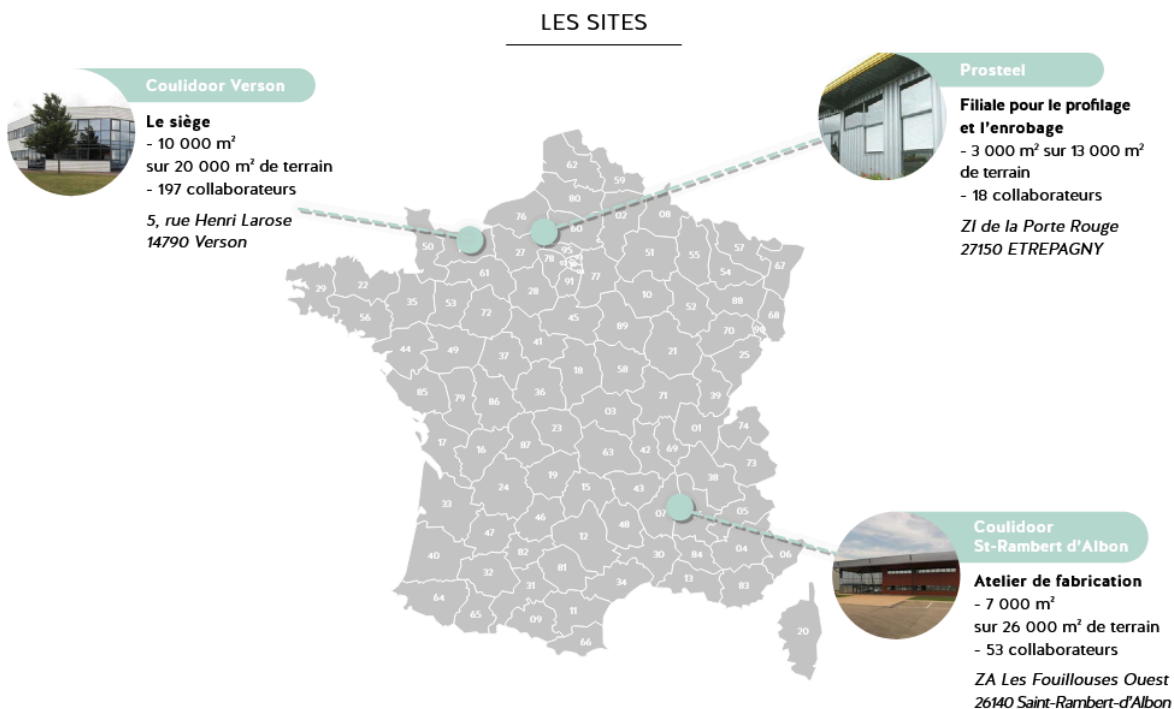


Figure 2 : Carte des sites de COULIDOOOR en France

L'entreprise Coulidoor possède différents sites répartis dans toute la France comme le montre la carte présentée en figure 2 mais c'est dans le site de Verson qui est le siège de COULIDOOOR dont M. FOUCHARD Guillaume est le directeur de site, que j'ai effectué mon stage. J'ai intégré l'équipe de Méthodes et Production dont le responsable est M. AL-JRAB Badredine qui est également mon tuteur durant ce stage.

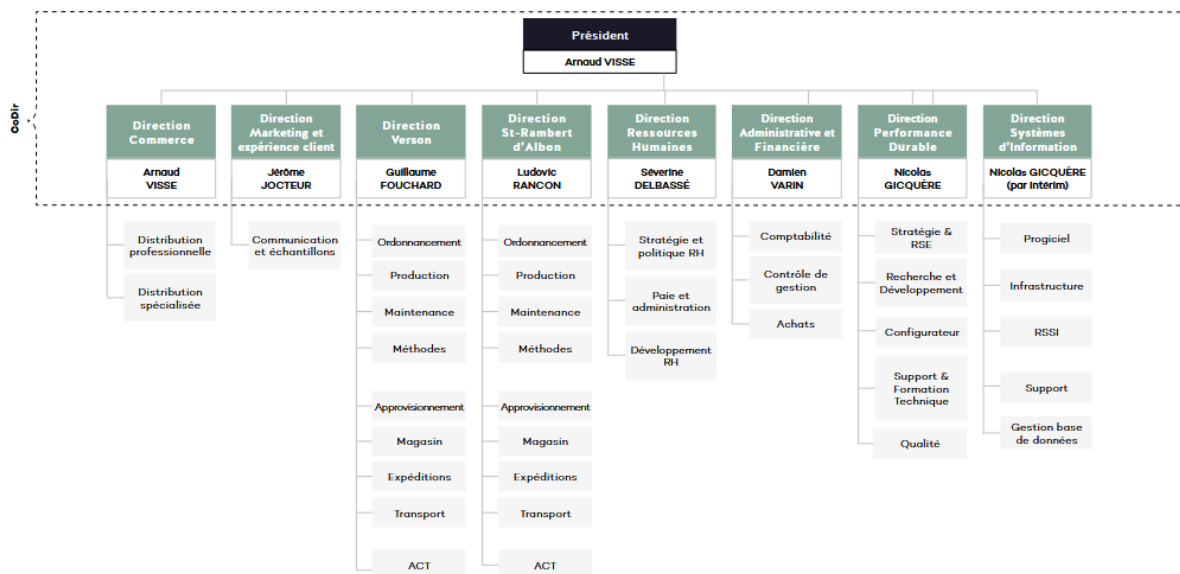


Figure 3 : Organigramme de l'équipe COULIDOOOR à Verson

Le site de Verson possède différentes équipes comme le montre l'organigramme. Durant ce stage, j'ai dû collaborer avec plusieurs de ces équipes, notamment avec l'équipe informatique pour avoir accès au réseau et pour avoir accès au matériel et aux logiciels nécessaires au bon déroulement du stage, je devais également collaborer avec cette équipe pour que mon système communique avec l'ERP qu'ils utilisent qui se nomme Silog.

De plus je devais collaborer avec l'équipe de maintenance pour qu'ils installent toutes les prises murales dont j'aurai besoin c'est-à-dire les prises secteurs ou les prises RJ45, ils étaient également responsables des commandes de composants que je devais effectuer. J'ai aussi dû m'organiser avec eux pour emprunter le matériel nécessaire à la mise en place de mon système comme un fer à souder, une pince à dénuder, des ciseaux, une rallonge électrique, des connecteurs wago et un décapeur thermique entre autres.

Enfin, j'ai intégré l'équipe de méthodes et production dont mon tuteur M. Badredine AL-JRAB est le responsable et avec laquelle j'ai collaboré pour installer les meubles et organiser la répartition des différents bacs, casiers ou tiroirs dans la nouvelle zone de préparation d'accessoires. Ces missions étaient essentielles pour comprendre l'organisation de la nouvelle zone de préparation d'accessoires et ainsi pouvoir concevoir l'organisation du système au sein de cette nouvelle zone. J'ai également collaboré avec les différents opérateurs de cette zone pour voir avec eux si le système leur convenait, s'il n'était pas trop restrictif, si les voyants étaient assez visibles en choisissant la bonne couleur. Enfin, tout au long du stage, j'ai eu l'occasion d'échanger avec mon tuteur de stage pour lui faire part de l'avancement de la mission et pour voir avec lui les différentes modifications à effectuer sur le système pour que celui-ci réponde aux attentes de l'entreprise.

Présentation de l'environnement de travail

Le lieu de travail

J'ai effectué la plus grande partie de mon stage sur le site de Verson, dans les locaux de COULIDOO^R. C'est ici qu'est installé le siège de COULIDOO^R et où les différentes équipes travaillent, notamment l'équipe informatique, l'équipe de maintenance, l'équipe méthodes, l'équipe de la communication, l'équipe RH et les équipes de production.

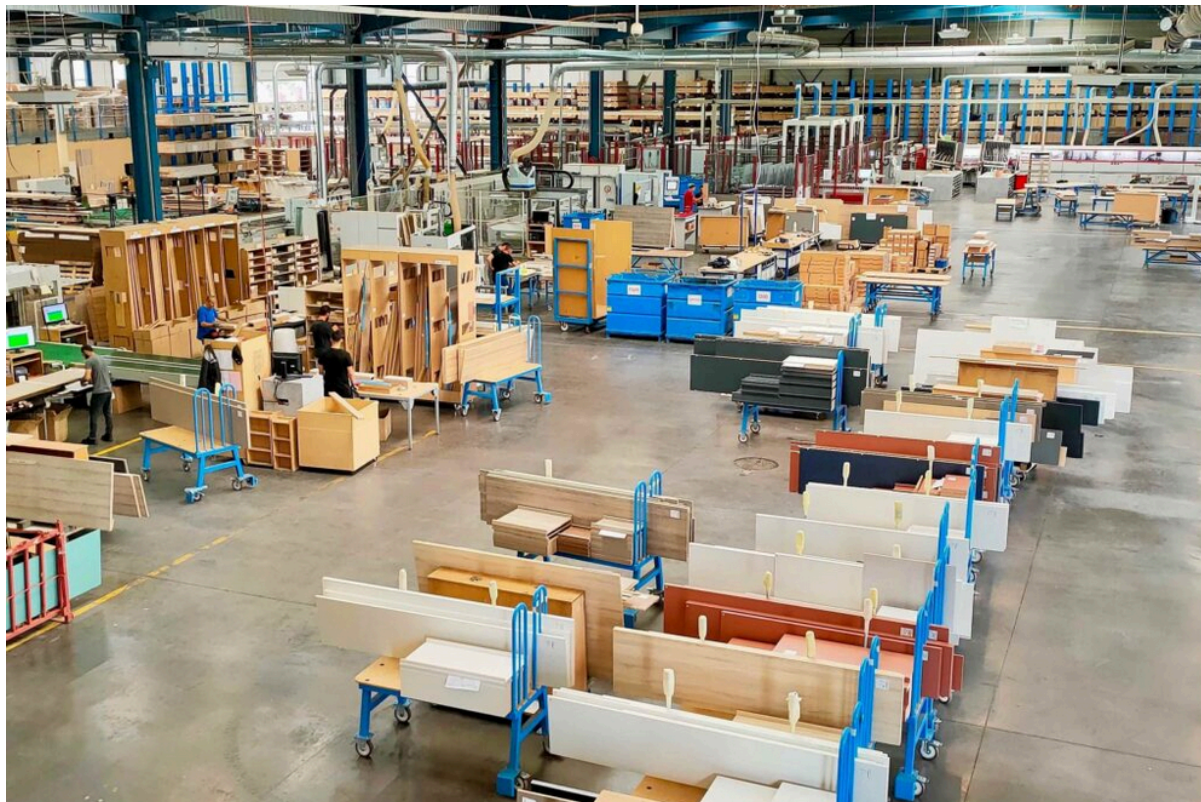


Figure 4 : Image du site de COULIDOO^R à CFR

J'ai travaillé dans un environnement en perpétuelle activité comme le montre la figure 4. Toute la journée, humains et machines sont en activité pour préparer les différentes commandes des clients comme le montre le cycle illustré par la figure 5. Les équipes y travaillent en 3 x 8 du lundi au vendredi.

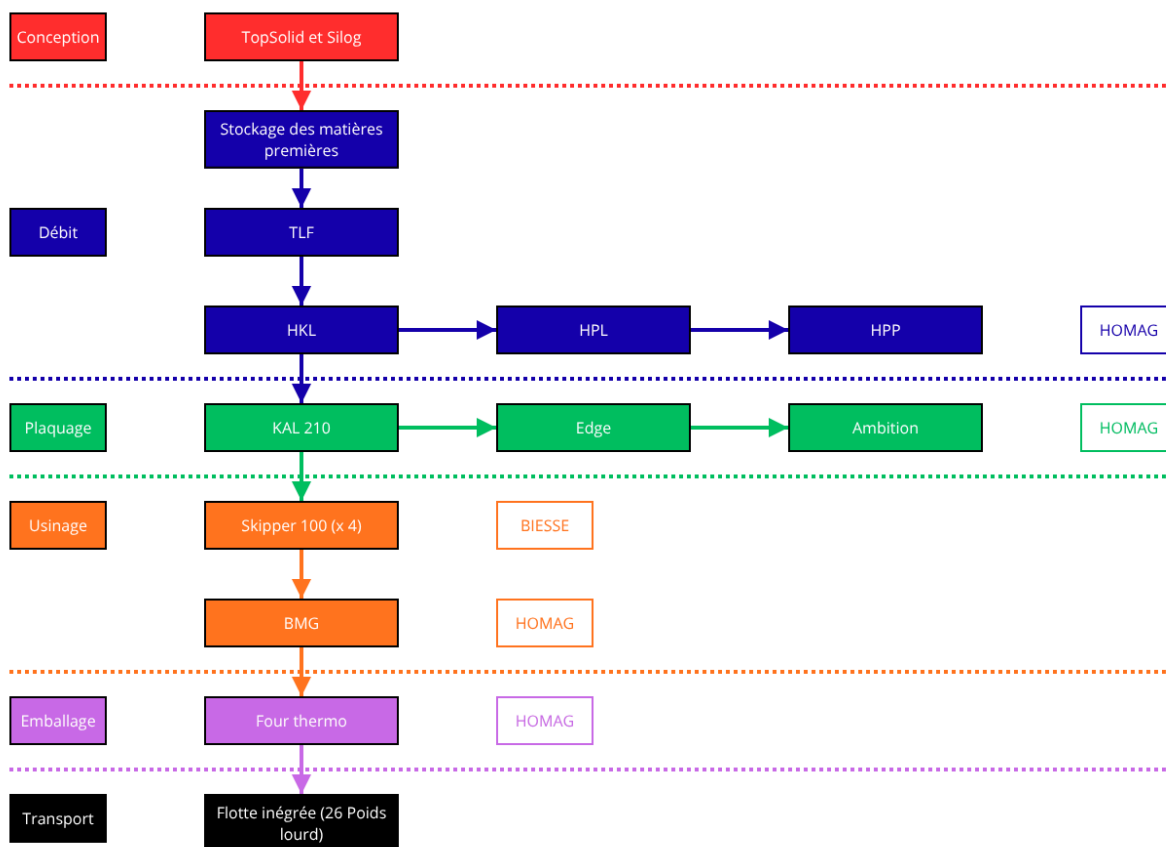


Figure 5 : Organigramme présentant la chaîne de production de Coulidoor

Tout d’abord, durant la première phase qui est la phase de conception, les meubles sont conçus sur le logiciel TopSolid et sont intégrés dans la base de données avec l’ERP Silog. Ensuite, il y a la phase de débit des panneaux de particules (bois reconstitué), durant laquelle les machines utilisent les matières premières qui sont stockées dans l’usine puis elles passent dans le TLF, dispositif permettant de transporter les panneaux de bois à l’aide de ventouses vers les machines de coupe qui sont la HKL, la HPL et la HPP, machines de la marque HOMAG, permettant de découper le panneau reçu en suivant un programme spécifique pour chaque commande à fabriquer. Ensuite, il y a la phase de plaquage durant laquelle le panneau précédemment découpé est plaqué ou habillé sur les côtés grâce aux machines suivantes : la KAL 210, la Edge et l’Ambition qui sont également des HOMAG. Suite à ça, il y a la phase d’usinage durant laquelle les panneaux découpés et plaqués suivent des opérations de perçage dans l’une des quatre machines de type “Skipper” disponible, qui sont des équipements de la marque BIESSE, puis dans le BMG qui est une machine produite par HOMAG. Enfin, il y a la phase d’emballage où les panneaux sont emballés grâce au four avec un film thermo-rétractable. Pour finir, ces meubles sont transportés grâce à la flotte de 26 camions intégrée et avec les conducteurs qui sont des salariés propres à l’entreprise.



Figure 6 : Image des EPI nécessaires sur site

Etant donné que les machines en activité font du bruit, des bouchons d'oreilles comme illustré sur l'image de gauche de la figure 6, doivent être mis pour protéger l'audition des opérateurs. De plus, des transpalettes et des chariots élévateurs circulent sur le site, c'est pourquoi il faut porter des chaussures de sécurité, comme illustré sur l'image de droite de la figure 6 à l'intérieur de l'entreprise.

Les journées de travail

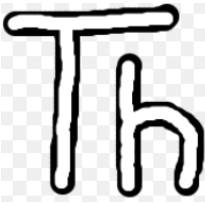
			
Google docs	Google sheets	Google slides	Gmail
			
Visual studio code	Solidworks	Thonny	Canva

Figure 7 : Images des logos des logiciels utilisés

Durant ma première journée de travail, l'entreprise m'a fourni un ordinateur de travail avec lequel j'ai travaillé tout au long du stage. Durant le stage, j'ai utilisé différents logiciels illustrés sur la figure 7.

J'ai utilisé google docs pour rédiger différents types de fichiers (compte-rendu hebdomadaire, compte-rendu de réunion, fichier d'accompagnement à la maintenance).

Google sheets était utile pour réaliser différents types de fichiers (fichier de commande, schéma de numérotation, planification, tâches à faire).

Google slides a servi pour réaliser des fichiers de présentation à l'entreprise. Pour présenter le système pour qu'il soit validé.

Gmail était le logiciel utilisé pour toutes les communications internes par mail qinsi que le chat instantané qui y est intégré..

Visual studio code m'a permis de créer une page web permettant de représenter virtuellement le système développé en ayant un suivi en temps réel de l'état des différentes pièces du système.

Les boîtiers 3D permettant d'installer les différentes pièces du système ont été conçus et réalisés à l'aide de Solidworks.

Le code pour la Raspberry Pi 5 et pour les différents ESP32 est issu du logiciel Thonny.

Enfin, les différents schémas représentant le système (schéma fonctionnel, schéma électrique, diagramme de fonctionnement) proviennent de Canva.

Les journées de travail étaient animées par différentes tâches :

- Avoir différentes réunions concernant l'avancée du projet
- Installer des meubles dans la nouvelle zone de préparation d'accessoires
- Effectuer des recherches pour construire l'architecture du système
- Effectuer de la soudure pour réaliser les différents bandeaux de leds nécessaires pour le système
- Réaliser des schémas électriques, fonctionnels du système
- Réaliser un fichier d'aide à la maintenance
- Réaliser une page web pour observer le fonctionnement du système virtuellement
- Réaliser des fichiers de commande
- Réaliser du code pour le système
- Réaliser un boîtier 3D pour installer le système
- Installation et tests du fonctionnement des bandeaux de leds
- Réaliser un tableau de bord pour avoir un suivi en temps réel de l'avancée de la préparation d'accessoires
- Réaliser des opérations de maintenance sur le système (pannes)
- Accompagner les opérateurs et résoudre ensemble des problèmes identifiés ou effectuer les améliorations souhaitées

Présentation des missions

Présentation générale du système Pick To Light

L'objectif principal du stage est de réaliser un système Pick To Light permettant aux opérateurs de la préparation d'accessoires d'avoir un détrompeur leur permettant d'avoir une aide pour la prise des articles présents sur le bon de préparation. Cette aide leur permet également de réduire leur charge mentale, ils ne vont plus avoir besoin de rechercher la référence par son numéro mais uniquement par observation du signal lumineux. Ma toute première mission était donc de concevoir l'architecture du système. J'ai donc commencé par effectuer des recherches sur des systèmes existants c'est-à-dire des systèmes Pick To Light déployés dans d'autres entreprises. J'ai retenu de cette analyse l'utilisation de leds et pour des questions d'optimisation avec le système demandé et les contraintes imposées, par exemple l'espace limité, je n'ai pas retenu l'utilisation des boutons et des écrans.

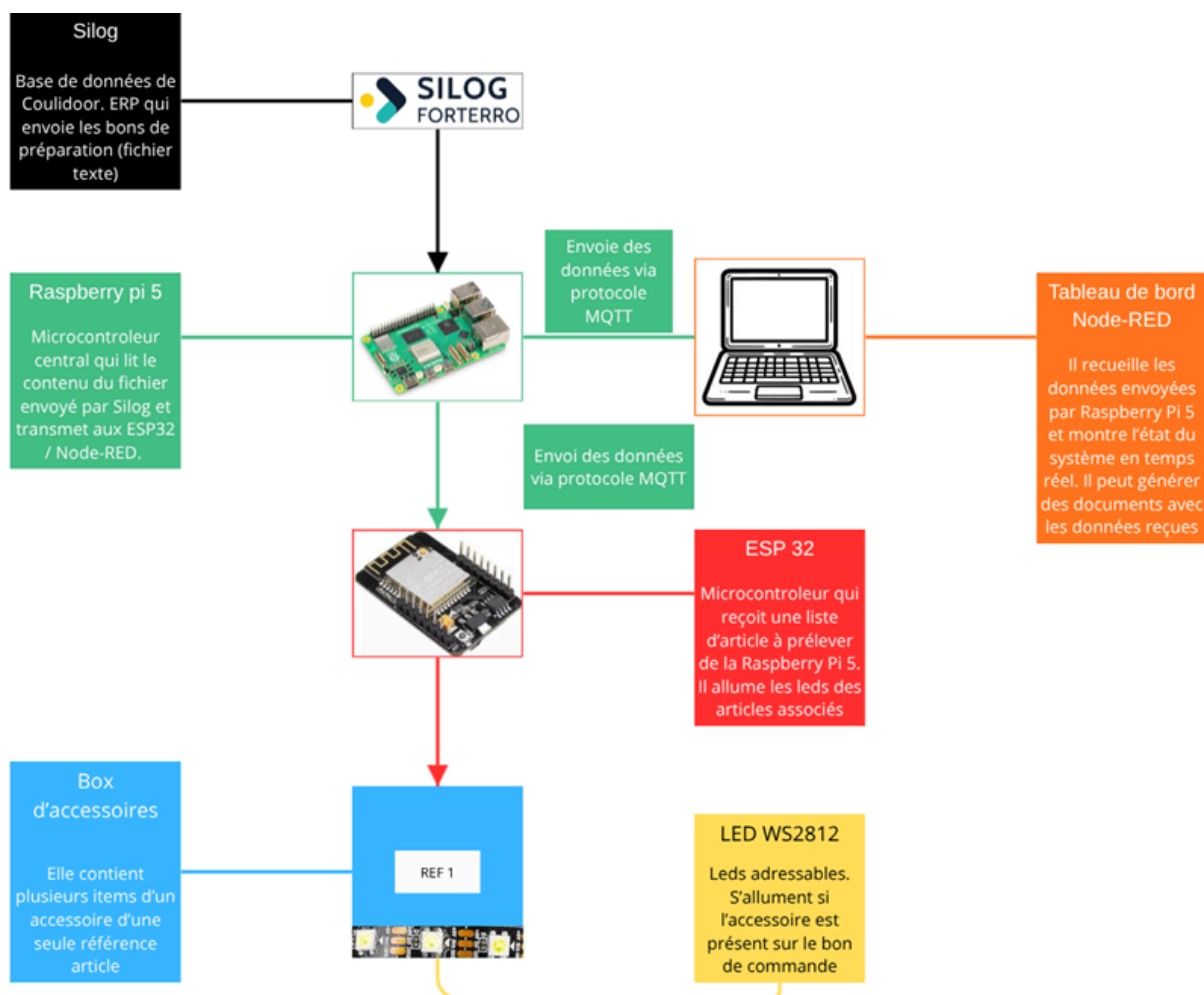


Figure 8 : Schéma fonctionnel du système Pick To Light

Après des recherches sur des solutions existantes j'ai conçu un système qui se décompose en 4 grandes parties comme l'illustre la figure 8.

Tout d'abord, il y a Silog qui est l'ERP de Coulidoor. C'est la base de données de Coulidoor qui conserve toutes les données sur les clients de Coulidoor, sur les meubles, les articles et autres. Il va être chargé de générer les bons de commande puis de les transmettre au Raspberry Pi 5 sous la forme d'un fichier texte. Il va transmettre au fur et à mesure un nouveau bon de commande à chaque fois que l'opérateur scanne le code article de l'article qu'il a prélevé qui est présent sur le bon de commande. Le nouveau bon de commande est le même que le précédent mais avec l'article prélevé en moins et ceci jusqu'à ce que le bon de commande soit terminé.

Ensuite, il y a la Raspberry pi 5 qui est le microcontrôleur central du système, il reçoit les bons de commande de la part de Silog, les lit puis transmet les articles demandés aux ESP32 associés en fonction de l'emplacement de l'article demandé. De plus, il transmet au tableau de bord Node-RED les différents articles présents sur le bon de commande, permettant de suivre en temps réel l'état du système réel avec un système virtuel.

Puis, il y a les ESP32 qui sont les microcontrôleurs chargés de l'allumage des différentes leds associées aux articles. Ils reçoivent les emplacements des leds à allumer de la part de la Raspberry Pi 5. Chaque ESP32 a un intervalle d'emplacement d'article qui lui est associé. Ils allument les leds des articles envoyés par la Raspberry Pi 5. Les leds sont des WS2812 qui sont des leds adressables permettant un montage en série et l'allumage de leds spécifiques sans avoir à allumer les leds précédentes.

Enfin, il y a le tableau de bord Node-RED qui est un tableau de bord sur lequel nous pouvons observer les différentes leds allumées. Il est également possible de réaliser des graphiques présentant le rythme de préparation des commandes. Il est également possible de réaliser des fichiers CSV présentant des statistiques sur la préparation des commandes.

Choix réalisés pour l'architecture du système



Figure 9 : Image de la Raspberry Pi 5

Pour l'architecture du système, il fallait un microcontrôleur capable de recevoir des données sur un bon de commande, capable de transmettre ces données et d'avoir accès à un fichier présent sur un serveur. Etant donné que j'avais déjà utilisé une Raspberry dans des projets personnels et que je savais qu'elle était capable de réaliser toutes ces tâches, j'ai opté pour une Raspberry Pi 5 illustrée par la figure 9. L'avantage proposé par cette carte

est également son OS permettant d'avoir accès à différents logiciels de programmation et à internet depuis la carte.



Figure 10 : Image d'un ESP32

Étant donné qu'on ne pouvait pas tirer des longs câbles dans la pièce, j'ai choisi de déporter mon système à plusieurs endroits de la pièce. L'idée était que les microcontrôleurs allaient recevoir les données de la Raspberry Pi, ce qui limite le câblage. Il fallait donc trouver des microcontrôleurs capables de recevoir les données. Avec des recherches, j'ai choisi les ESP32, illustrés par la figure 10 car ils sont bon marché, bien documentés et sont souvent utilisés dans des projets présentés sur internet. J'ai choisi d'en utiliser 8 pour contrôler localement l'allumage des leds et pour limiter le câblage.

Pour la réalisation technique, j'avais initialement décidé de réaliser les différents codes nécessaires avec le langage C mais ceci posait problème pour utiliser des bibliothèques pour les bandeaux de leds avec la Raspberry Pi 5 qui est plus restrictive que ses prédécesseurs. J'ai donc choisi de basculer sur le langage Python qui est un autre langage informatique que je maîtrise et qui est très utilisé également par les développeurs informatiques, c'est un langage qui est maîtrisé par l'équipe informatique interne à la société. Un autre avantage de ce langage est qu'il est possible de coder les ESP32 avec du micro-python donc seules quelques modifications sont nécessaires pour passer du code de la Raspberry Pi 5 vers les ESP32.

De plus, il fallait que les ESP32 communiquent avec la Raspberry Pi 5, j'ai donc choisi d'utiliser le protocole MQTT car je l'ai déjà utilisé dans des projets personnels et celui-ci permet de réaliser les tâches demandées.



Figure 11 : Image d'un bandeau de leds ws2812

Il était nécessaire d'ajouter au système un signal visuel, c'est pourquoi j'ai opté pour des leds. Mais celles-ci ne devaient pas être de simples leds, il fallait qu'elles soient adressables car seules les leds associées aux articles présents sur le bon de commande doivent s'allumer. Il fallait donc qu'elles puissent être allumées en totale indépendance. Avec des recherches j'ai opté pour les leds WS2812 illustrées par la figure 11 car elles sont adressables et des bibliothèques permettant de les contrôler sont disponibles sur internet.



Figure 12 : Image de connecteurs JST-SM

Pour réaliser les bandeaux de leds, j'ai décidé d'utiliser des connecteurs JST-SM illustrés par la figure 12, car ceux-ci permettent une bonne connexion des petits bandeaux de leds entre eux, de plus la longueur des bandeaux est réglable avec ces connecteurs. Enfin, le grand avantage de ces connecteurs est la modifiabilité du système car si un bandeau présente une défaillance matérielle, il suffit de déconnecter les connecteurs et de remplacer le bandeau par un nouveau en insérant les connecteurs. Il n'y a pas besoin d'enlever une ligne de bandeau, le système est modifiable localement. Ceci veut dire aussi qu'il est possible d'ajouter des bandeaux au bout de la ligne de bandeau, il est donc possible d'ajouter des articles.

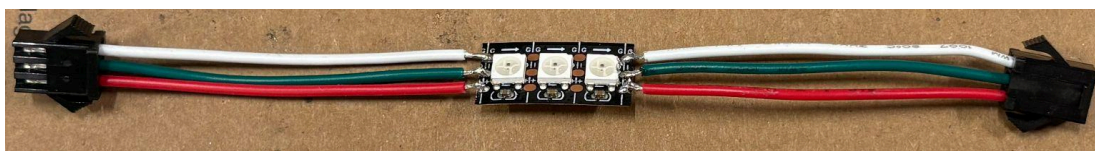


Figure 13 : Image d'un bandeau de leds

Pour réaliser les bandeaux de leds avec les connecteurs JST-SM, il était nécessaire de souder les trois fils aux extrémités des petits bandeaux de leds découpés comme l'illustre la figure 13. La longueur des câbles était donc modulable.

Il a été vu avec l'entreprise que le système devait fonctionner avec Silog, c'est-à-dire que le système doit allumer toutes les leds associées aux articles à prélever du bon de commande, pour ce faire, Silog doit générer un fichier texte respectant le format défini qui sera décrypté par la Raspberry Pi 5, l'opérateur doit scanner le code article de l'article qu'il a prélevé sur le bon de commande, l'information est récupérée par Silog qui doit renvoyer un nouveau bon de commande en ayant enlevé l'article déjà prélevé et ce jusqu'à la fin du bon de commande avant d'en commencer un nouveau.

Enfin je souhaitais pouvoir suivre virtuellement l'état des différentes leds du système, c'est pourquoi j'ai créé dans un premier temps une page web permettant de montrer les leds allumées en temps réel. Dans des projets personnels, j'avais créé des tableaux de bord à l'aide de Node-RED. C'est quelque chose que je souhaitais réaliser de nouveau, c'est pourquoi j'ai alimenté mon tableau de bord Node-RED avec le code réalisé pour la page web ce qui permettait d'avoir un tableau de bord bien abouti pour monitorer le fonctionnement.

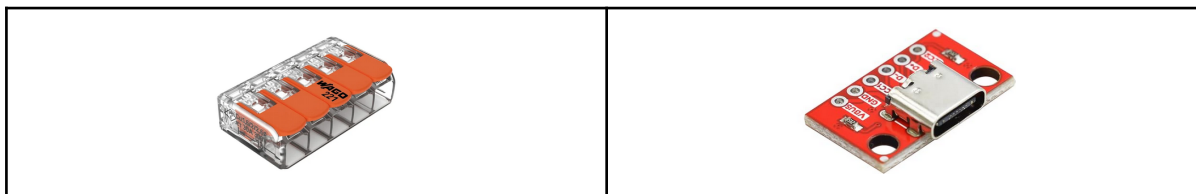


Figure 14 : Image d'un connecteur Wago et d'un USB-c breakout

Pour alimenter le système, il a été vu avec l'entreprise qu'il fallait limiter les alimentations, l'architecture a donc été pensée pour utiliser 3 alimentations réparties dans toute la zone. Pour partager le courant, j'ai utilisé des connecteurs wago illustrés par l'image de gauche de la figure 14. Ceux-ci permettent d'avoir la même tension dans les différentes fentes du connecteur. Avec des fils, l'alimentation peut être partagée dans les différentes parties du système. J'ai dû également trouver des fils capables de supporter un fort ampérage pour alimenter tout le système. Et j'ai dû utiliser des USB-c breakouts illustrés par l'image de droite de la figure 14, pour adapter une sortie en USB-c en filaire.



Figure 15 : Image des leds avec les profilés et les caches opaques

Enfin, pour protéger le système, il a été vu avec l'entreprise qu'il fallait installer des profilés en aluminium pour installer les leds. Pour les protéger et pour diminuer l'intensité du faisceau, il a été vu qu'il fallait mettre des caches opaques devant les leds comme le montre la figure 15. De plus, pour protéger les microcontrôleurs et le câblage il a été vu qu'il fallait acheter un boîtier 3D pour protéger la Raspberry Pi 5 et concevoir et réaliser des boîtiers 3D pour les ESP32.

Cette architecture est issue des propositions effectuées à l'entreprise, des échanges autour de ce système et des améliorations demandées par l'entreprise.

Fonctionnement du système

Durant ce stage, après avoir conçu l'architecture du système, je devais réaliser le développement hardware permettant au système de fonctionner.

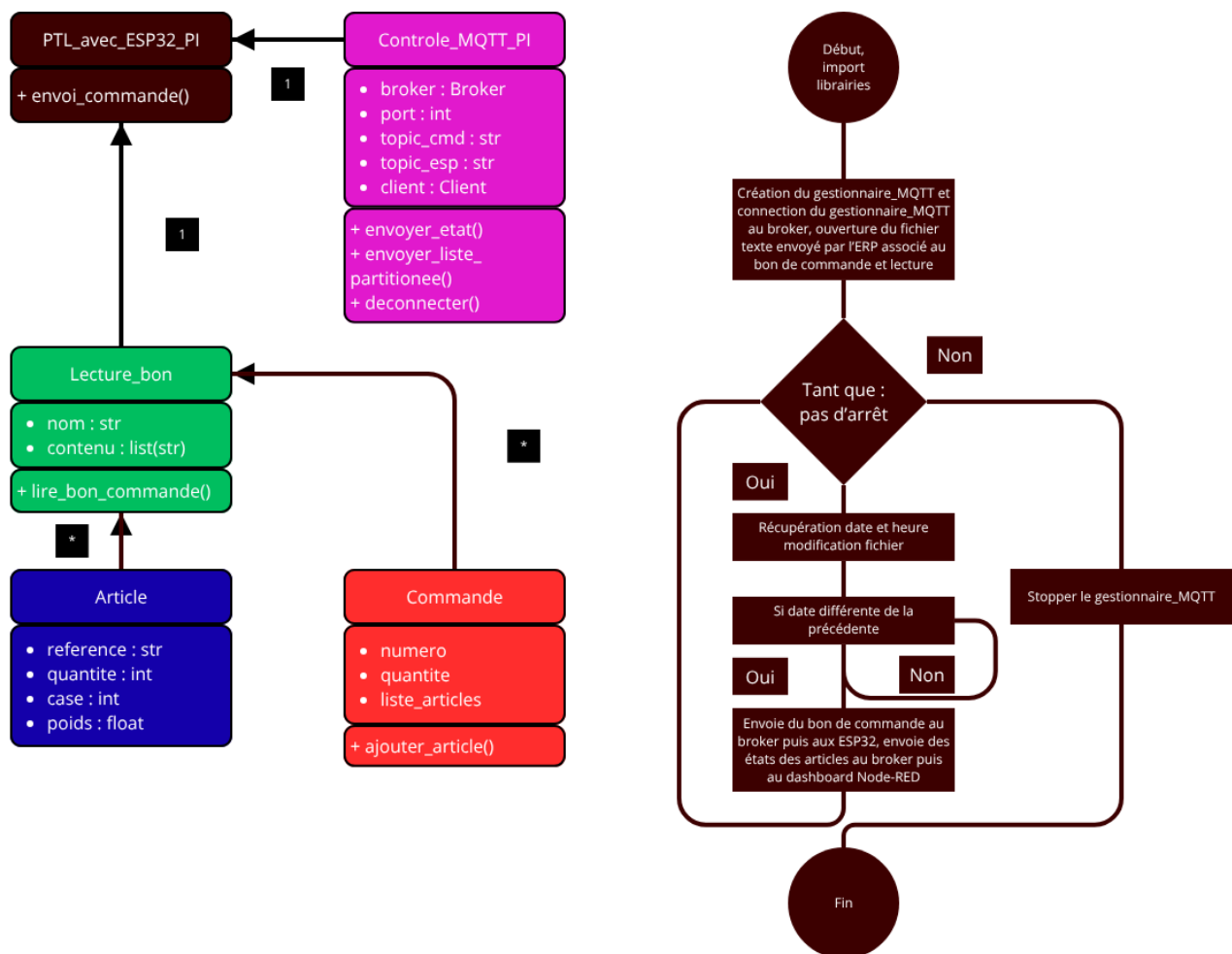


Figure 16 : Organigramme du fonctionnement du Raspberry pi 5

La Raspberry Pi 5 qui est la carte principale chargée de la réception du bon de commande, de sa lecture et de sa transmission vers les différents ESP32 opère comme le décrit la figure 16. Tout d'abord, le programme principal de la Raspberry Pi 5 est alimenté par différentes classes. Le format de classe permet l'organisation du code et la modification locale du code ainsi qu'une réutilisation et une modification simplifiée de ces classes.

La première s'appelle **Controle_MQTT_PI**, elle possède différents attributs comme le broker, le port, les topics des ESP32 et du tableau de bord Node-RED et un client. Ceux-ci sont des attributs nécessaires pour le protocole MQTT et vont permettre la communication avec les différents ESP32 et avec le tableau de bord Node-RED. Elle possède également

différentes fonctions. La fonction `envoyer_etat()` permet d'envoyer au tableau de bord Node-RED des données sur les leds qui vont être allumées, la fonction `envoyer_liste_partitionnee()` permet d'envoyer aux différents ESP32 les leds à allumer (chaque ESP32 possède un intervalle de 100 emplacements à leur charge) et la fonction `deconnecter()` permet au Raspberry Pi 5 de se déconnecter du tableau de bord Node-RED.

La deuxième s'appelle `Lecture_bon`, elle possède deux attributs, le nom du bon de commande et une liste permettant de recueillir le contenu du bon de commande. Elle possède une fonction qui s'appelle `lire_bon_commande()` qui a pour but de lire le fichier texte envoyé par Silog et qui stocke les emplacements des articles à prélever. Cette classe utilise plusieurs instances d'autres classes. La première est `Article`, elle possède plusieurs attributs comme la référence de l'article à prélever, sa quantité, son emplacement et son poids. Cette classe a pour but de représenter l'article à prélever. Il y a également la Classe `Commande`, celle-ci possède plusieurs attributs comme le numéro de la commande, le nombre d'articles présents sur le bon de commande et une liste d'articles à prélever. Cette classe possède la fonction `ajouter_article()` qui permet d'ajouter un article dans la liste d'articles à prélever.

Ces deux classes sont utilisées dans le programme principal qui s'appelle `PTL_avec_ESP32_PI`, celui-ci ne possède pas d'attributs et utilise une instance de `Lecture_bon` lui permettant de lire le bon de commande reçu de la part de Silog et une instance de `Controle_MQTT_PI` lui permettant de communiquer avec le tableau de bord Node-RED et avec les différents ESP_32. Il ne possède pas d'attributs mais possède une fonction qui s'appelle `envoi_commande()` qui a pour but d'envoyer aux différents ESP32 les emplacements des leds à allumer.

Le programme principal fonctionne de la manière suivante : il importe les librairies nécessaires à son bon fonctionnement, il crée le `Gestionnaire_MQTT` qui va lui permettre de communiquer avec les ESP32 et le tableau de bord Node-RED. Il lit ensuite le bon de commande envoyé par Silog. Tant qu'il n'y a pas d'arrêt demandé, le programme vérifie si le fichier a été modifié. Si c'est le cas, il envoie aux différents ESP32 les emplacements des leds à allumer et au tableau de bord Node-RED l'emplacement des leds qu'il faut allumer. Sinon il attend que le fichier soit modifié. Enfin, si un arrêt est demandé, il coupe la communication avec les ESP32 et le tableau de bord Node-RED et le programme se termine.

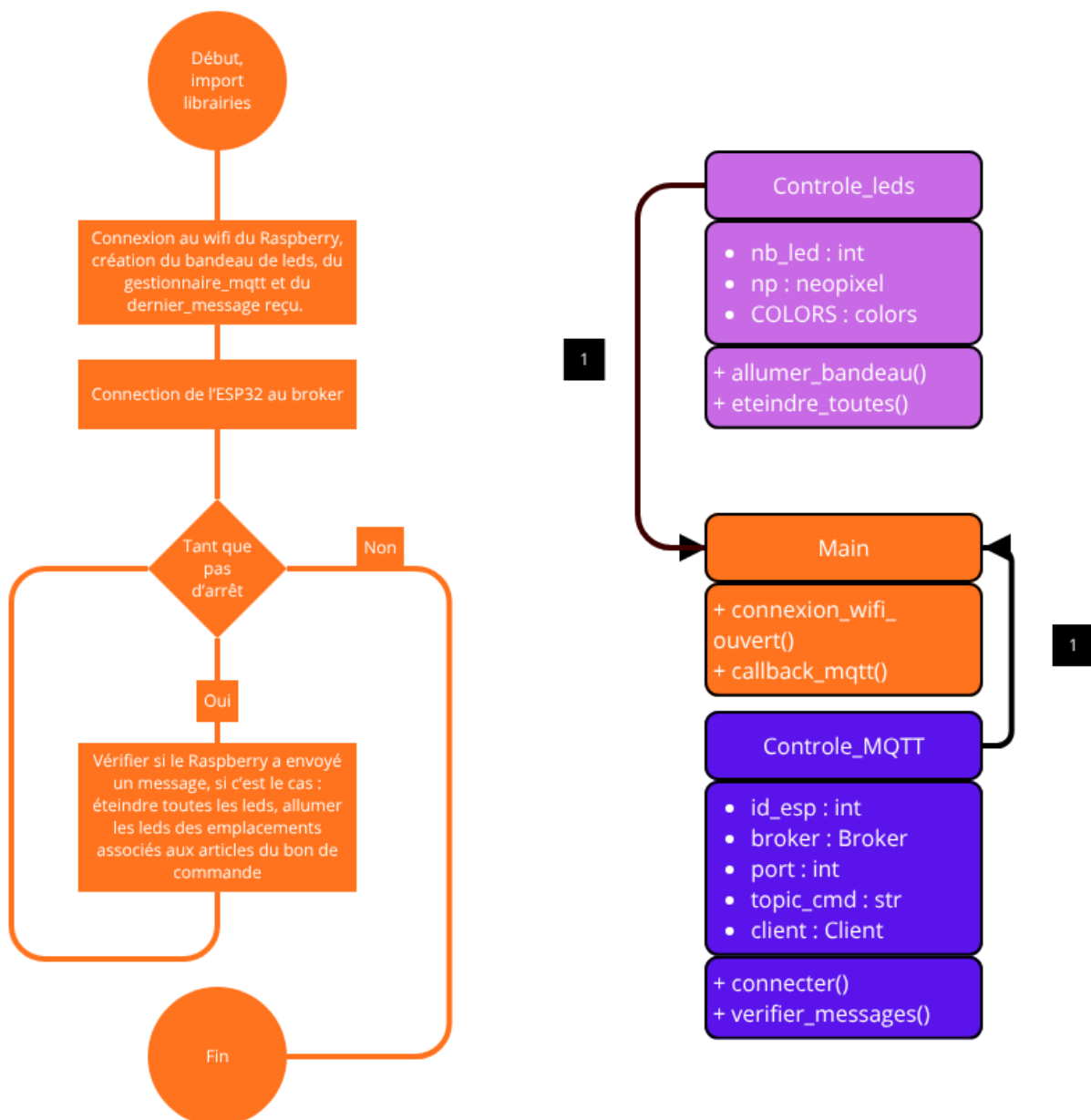


Figure 17 : Organigramme du fonctionnement des ESP32

L'ESP32 qui est la carte chargée de la réception des données envoyées par la Raspberry Pi 5 opère comme le montre la figure 17. Tout d'abord, le programme principal de l'ESP32 est alimenté par différentes classes. Le format de classe permet l'organisation du code et la modification locale du code ainsi qu'une réutilisation et une modification simplifiée de ces classes.

La première s'appelle `Controle_leds`, elle possède différents attributs comme le nombre de leds, les couleurs et un neopixel définissant le port et le nombre de leds connecté avec l'ESP32. Elle possède également différentes fonctions. La fonction `allumer_bandeau()` permet d'allumer toutes les leds présentes sur le bandeau associé à un

article à prélever, la fonction `eteindre_toutes()` permet d'éteindre toutes les leds du long bandeau de leds.

La deuxième s'appelle `Controle_MQTT`, elle possède différents attributs comme le numéro de l'ESP32, le broker, le port et le topic de la Raspberry Pi 5 et un client. Ceux-ci sont des attributs nécessaires pour le protocole MQTT qui vont permettre la communication avec la Raspberry Pi 5. Elle possède également différentes fonctions. La fonction `connecter()` permet de se connecter avec la Raspberry Pi 5, la fonction `verifier_messages()` permet de vérifier si la Raspberry Pi 5 n'a pas envoyé des articles à prélever.

Ces deux classes sont utilisées dans le programme principal qui s'appelle "main", celui-ci ne possède pas d'attributs et utilise une instance de `Controle_leds` lui permettant d'allumer et d'éteindre les leds du bandeau connecter à l'ESP32. Il utilise également une instance de `Controle_MQTT` lui permettant de communiquer avec la Raspberry Pi 5. Il ne possède pas d'attributs mais possède deux fonctions, l'une s'appelle `connexion_wifi_ouvert()` et a pour but de permettre à l'ESP32 de se connecter au wifi du Raspberry Pi 5, l'autre fonction s'appelle `callback_mqtt()` et permet d'allumer les leds qu'il faut allumer dès lors que l'ESP32 reçoit un bon de commande de la part de la Raspberry Pi 5.

Le programme principal fonctionne de la manière suivante : il se connecte à la Raspberry Pi 5, il importe les librairies nécessaires à son bon fonctionnement, il crée le `Gestionnaire_MQTT` qui va lui permettre de communiquer avec la Raspberry Pi 5 et il crée le bandeau de leds lui permettant d'allumer et d'éteindre les différentes leds du bandeau. Il vérifie ensuite si l'ESP32 a reçu un message et si c'est le cas, il éteint toutes les leds du bandeau puis allume les leds associées aux articles présents sur le bon de commande envoyé par la Raspberry Pi 5.

Alimentation du système

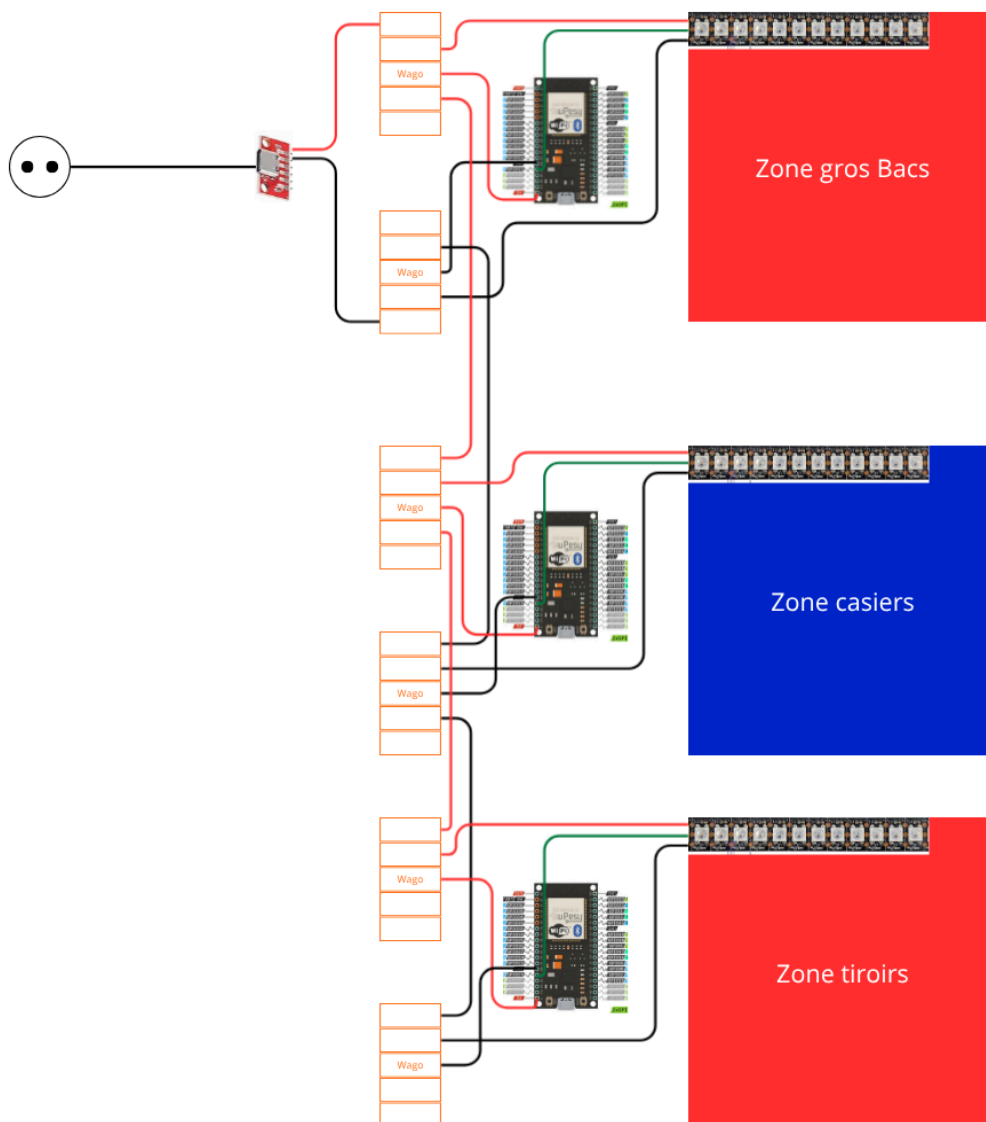


Figure 18 : Schéma représentatif du câblage du système

Pour alimenter le système et limiter les alimentations, le système est alimenté comme le montre la figure 18. Une alimentation est connectée à la prise secteur et son embout USB-c va dans un USB-c breakout qui convertit le type USB-c en filaire. Ensuite, le +5V de l'USB-c breakout va dans un connecteur wago qui va distribuer le +5V au premier ESP32 et au bandeau de leds qui lui est associé. En même temps, la masse de l'USB-c va dans un connecteur wago qui va distribuer la masse au même ESP32 et au même bandeau de leds. Il y a également un câble qui va du premier wago +5V vers le deuxième wago +5V et un câble qui va du premier wago masse vers le deuxième wago de masse et ces deux wago vont alimenter le deuxième ESP32 et le bandeau de leds qui lui est associé et c'est le même principe pour alimenter les troisièmes wago et donc le troisième ESP32 et le bandeau de leds associés. C'est le même câblage pour la partie droite de la pièce.

Organisation de la pièce

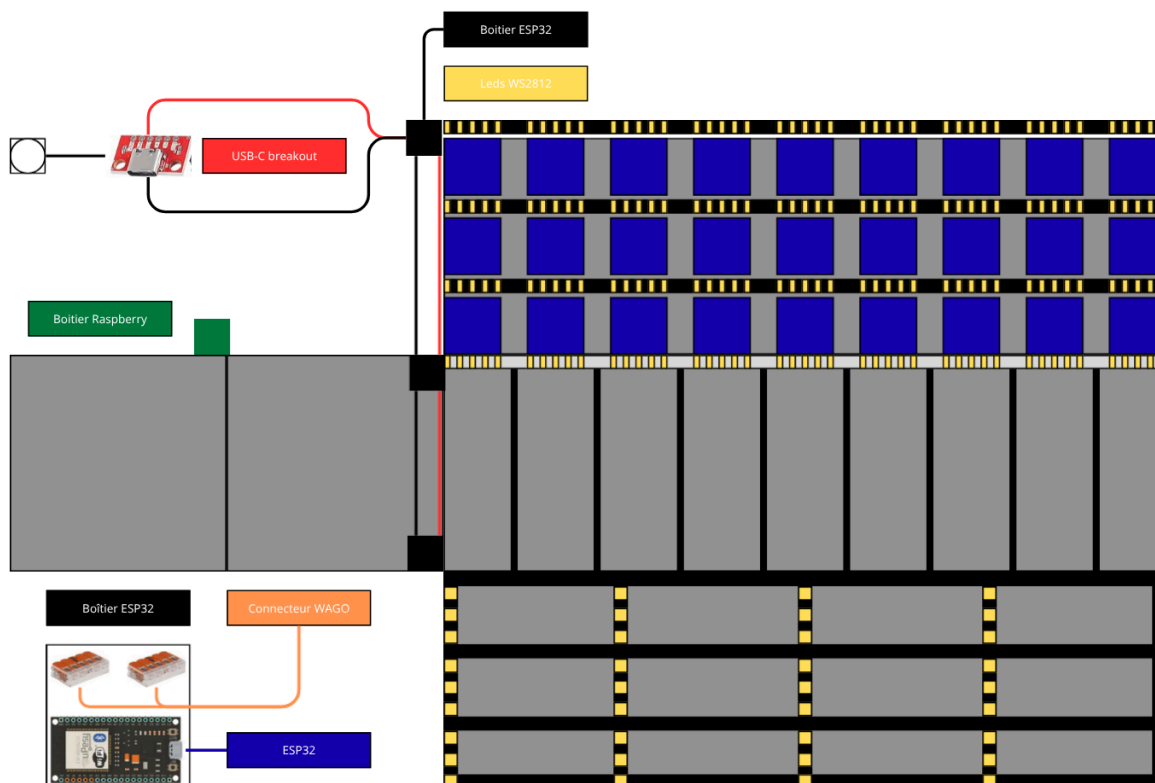


Figure 19 : Schéma représentatif de la pièce

La pièce est décomposée en deux côtés, le côté droit et le côté gauche qui sont similaires mais avec quelques différences mineures, cette organisation est présentée par la figure 19. Il y a tout d'abord le câblage présenté en figure 18, les connecteurs wago et les ESP32 qui vont être stockés dans des boîtiers 3D et les petits bandeaux de leds sont reliés en série et partent du bandeau qui est relié à l'ESP32. Il y a trois longs bandeaux opérant ainsi, celui pour les gros bacs, celui pour les casiers et celui pour les tiroirs.



Figure 20 : Image de la pièce avec le système en fonctionnement

La pièce montrée en figure 20 est représentée par la figure 19. Celle-ci est composée de deux côtés. Le côté gauche possède une zone avec des grands bacs, une zone avec des casiers juste en dessous et une zone avec des tiroirs intégrés en partie basse, sous le plan de travail. Il y a également un plateau roulant présent sur la zone des casiers. C'est le même principe pour le côté droit, sauf que la zone des grands bacs, à la place d'avoir 3 lignes de grands bacs, elle possède une ligne composée de petits bacs (cet ajustement était nécessaire pour permettre aux opérateurs d'avoir accès à tous les casiers). Enfin, la zone des casiers est divisée en 3 sections. De plus, il y a une zone de petits bacs qui n'est pas présente sur le côté gauche.

Tableau de bord Node-RED



Figure 21 : Image du tableau de bord Node-RED

Ayant déjà créé des tableaux de bord Node-RED dans des projets personnels, je souhaitais réitérer le travail pour ce projet. J'y trouvais également une utilité pour le monitoring du système. J'ai donc commencé par créer une page web avec le langage HTML et CSS pour régler l'aspect du tableau de bord. J'ai ensuite utilisé ce code pour l'intégrer dans le tableau de bord Node-RED comme l'illustre la partie gauche de la figure 21. Celui-ci sert pour avoir un visuel en temps réel des articles à prélever sur le site. J'ai également ajouté des fonctionnalités avec les données qui circulent comme une jauge qui permet d'avoir le pourcentage d'avancée du bon de préparation, j'ai également ajouté des textes indiquant le nombre d'article restant à prélever présent sur le bon de préparation, le temps moyen pris par l'opérateur pour prélever une référence et avec ce temps moyen le temps prévu pour finir le bon de préparation et un indicateur de l'état de la commande si elle est commencée, en cours ou terminée. J'ai aussi mis deux leds indiquant si la prise d'une référence prend plus de temps que prévu et si l'opérateur est bloqué dans le cas où il n'a prélevé aucune référence depuis un certain temps. Enfin il y a un graphique présentant le temps pris par l'opérateur pour prélever un article en fonction de l'heure actuelle.

Ces données peuvent être utilisées pour générer des fichiers CSV permettant d'analyser la récurrence des articles prélevés, le rythme de picking. Ceci peut également servir pour anticiper les ruptures de stock possible pour commander en avance les articles.

Difficultés rencontrées et solutions apportées

Durant ce stage, j'ai été confronté à quelques difficultés. Tout d'abord, le premier système que j'avais imaginé et proposé ne comportait que la Raspberry Pi 5 mais imposait beaucoup de câblage. Ce qui posait problème avec la structure de la zone de préparation d'accessoires, avec l'alimentation des composants du système et avec sa protection dans la zone. C'est pourquoi j'ai proposé à l'entreprise de décomposer le système en plusieurs sous-systèmes avec les ESP32 qui reçoivent les données du Raspberry Pi 5 ce qui permet de limiter le câblage et de répartir les tâches. De plus, une panne n'impacte pas les autres parties du système ce qui leur permet de fonctionner indépendamment des autres.

Le deuxième problème venait d'un souci de bibliothèque pour les leds pour les contrôler avec le langage C, c'est pourquoi j'ai décidé de les contrôler avec le langage Python car la bibliothèque était accessible par la Raspberry Pi 5. D'autant plus que le code des ESP32 est en Micro-Python et donc nécessite uniquement quelques adaptations pour que le code soit compatible. C'est un langage que je maîtrise également donc basculer sur ce langage n'a pas posé problème.

Ensuite, pour éviter d'avoir à acheter une alimentation pour chaque ESP32 et pour alimenter les différents bandeaux de leds, j'ai réfléchi à une architecture d'alimentation composée d'une alimentation, d'un USB-c breakout, de câbles et de connecteurs wago qui permettent de distribuer le courant dans tout le système. C'était la première fois que je réalisais ce genre d'alimentation, j'ai donc fait des erreurs qui m'ont bloqué temporairement comme des mauvaises soudures et des fils pas assez dénudés mis dans les connecteurs wago qui empêchaient le système d'être alimenté et donc de fonctionner correctement.

L'entreprise me demandait de créer un système qui soit facilement modifiable, c'est-à-dire qui ne nécessite pas l'arrêt complet du système pour le réparer et qui permette de remplacer rapidement les pièces défectueuses du système. C'est pourquoi j'ai imaginé la création de petits bandeaux de leds soudés avec des connecteurs JST-SM dont je peux régler la longueur et permettent de remplacer un bandeau par un autre rapidement. Ceci rend le système plus souple mais a nécessité beaucoup de travail de soudure, 6 soudure par bandeau pour 300 bandeaux ce qui a pris pas mal de temps.



Figure 22 : Image d'un bandeau de leds avec un cache thermique

J'ai été exposé à un problème pour les leds accrochées sur les tiroirs, il fallait faire passer les câbles en dessous du meuble, j'ai donc dû souder des câbles aux connecteurs initiaux pour les allonger comme illustré sur la figure 22, le problème était que les soudures se touchaient, j'ai donc utilisé des caches thermiques avec un décapeur thermique pour isoler les différents câbles et je les ai ensuite accrochés en dessous du meuble.

La Raspberry Pi 5 devait avoir accès au fichier généré par Silog, c'est pourquoi j'ai dû collaborer avec l'équipe informatique pour avoir les droits d'accès à un serveur dédié, sur lequel le fichier allait être déposé par Silog. De plus, pour avoir accès aux emplacements des leds à allumer, j'ai demandé à l'équipe informatique de générer un fichier texte respectant un certain format pour me permettre d'avoir accès à la donnée souhaitée. Pour détecter la présence d'un nouveau bon de commande, j'ai dû adapter mon code pour qu'il détecte la modification du fichier envoyé en analysant la dernière date de modification du fichier. De plus, il fallait limiter l'accès aux codes et aux données pour une question de sécurité et pour simplifier les tâches des opérateurs c'est pourquoi j'ai modifié les paramètres de la Raspberry Pi 5 pour qu'elle se connecte directement à son WiFi et pour qu'elle lance automatiquement le programme lors de son démarrage.

Laisser les bandeaux de leds à même les meubles pouvait poser problème car plusieurs bandeaux pourraient être arrachés. C'est pourquoi, nous avons vu avec l'entreprise pour mettre les bandeaux dans des profilés en aluminium avec des caches opaques devant permettant de protéger les leds et de limiter leur intensité. Il fallait également enlever les languettes des connecteurs JST-SM car celles-ci ne rentraient pas dans les profilés. De plus, les câbles d'alimentation sont également protégés par des profilés et des caches ou par des goulottes PVC.

Durant les tests, il est apparu que le système fonctionnait anormalement, j'avais associé tous les ESP32 au même canal de communication, ce qui posait problème car seuls certains ESP32 recevaient les données des leds à allumer. J'ai donc dû décomposer ce canal de communication en plusieurs canaux avec chaque ESP32 qui a son canal de communication qui lui est associé. Ceci a résolu le problème car après cette modification, les différents ESP32 allumaient les leds des articles présents sur le bon de commande. J'ai également eu un souci quand j'essayais d'allumer toutes les leds du système car les dernières leds n'avaient plus assez de courant pour être alimentées à cause de la chute de tension. J'ai donc testé le fonctionnement du système en allumant la moitié des leds et le système fonctionnait correctement, les leds étaient allumées avec une bonne intensité. De plus, allumer la moitié des leds est suffisant car ceci correspond au nombre maximal d'articles présent sur un bon de commande.

Je voulais avoir un tableau de bord qui ait le même aspect qu'une page web, j'ai donc dû me former sur la création des pages web et j'ai dû apprendre à les adapter sur Node-RED pour avoir l'aspect attendu.

Enfin, l'entreprise avait la volonté d'ajouter un mot de passe au Wifi émis par la Raspberry mais à cause d'un souci de compatibilité entre la connection réseau des ESP32

avec le Wifi du RaspberryPi 5 avec le mot de passe, nous avons décidé de rester sur la version du Wifi sans mot de passe qui était elle fonctionnelle.

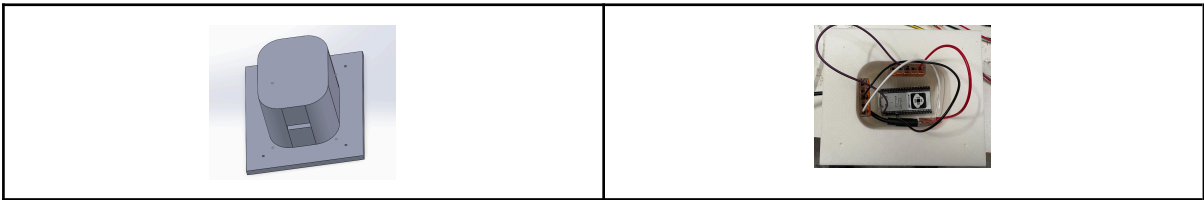


Figure 23 : Image du boîtier 3D avec les composants alimentation à l'intérieur

Je devais protéger les différents composants de mon système d'alimentation, c'est pourquoi j'ai conçu et imprimé en 3D un boîtier qui permet de contenir l'USB-c breakout, les connecteurs wago, l'ESP32 et les différents câbles d'alimentation comme l'illustrent les images de la figure 23.

Une fois que le système a été mis en fonctionnement, j'ai dû effectuer la maintenance moi-même, grâce aux dysfonctionnement détectées par les opérateurs, j'ai ainsi changé les bandeaux de leds défaillants, réglé les soucis de câblage des ESP32, j'ai transmis la liste des bandeaux de leds non identifiés sur l'ERP Silog et le système a eu de nouveau accès au fichier généré par Silog après avoir modifié le mot de passe expiré. Mais le plus gros problème venait lorsque Silog avait un nombre supérieur ou égal à 1000 à générer sur le bon de commande car il générait par défaut un espace qui était considéré comme un caractère spécial par mon système, empêchant la lecture du fichier et arrêtant le programme. La modification de ce fonctionnement a permis d'éviter les arrêts brutaux du système et permis aux opérateurs d'éviter de débrancher puis rebrancher les alimentations.

Pour préserver le système et pour éviter de possibles surchauffes, j'ai instauré une procédure d'allumage à chaque prise de poste de l'opérateur et à chaque retour de pause, il y a également une procédure d'extinction à chaque fin de poste ou à chaque départ en pause, permettant au système de refroidir et de ne pas être 24/24 allumé.

Coût total et bénéfices

Nom	Prix unitaire	Quantité	Total	Source	Image	Lien	Nom	Prix unitaire	Quantité	Total	Source	Image	Lien
Leds WS2812	24,99	10	249,9	Amazon		rd_w-xfoul&	Transformateur 5 V	13,54	3	40,62	RS Components		web/p/adaptateur
Carte Raspberry Pi 5 4GB	90,39	1	90,39	Amazon		d61NsNM9B	Carte ESP32	16,76	8	134,08	RS Components		de-developpeme
Connecteur JST-SM	9,99	15	149,85	Amazon		6ZPFDAM40	Câbles HDMI vers micro HDMI	3,73	1	3,73	RS Components		web/p/cables-rasp
USB-C breakout	9,29	1	9,29	Amazon		ltfIs8Ybv7ns	Alimentation Raspberry Pi	18,65	1	18,65	Amazon		vyvQ6bTC5gPp
Boîtier raspberry	15,99	1	15,99	Amazon		0rWwQCU0Z	Câbles femelle-femelle	3,29	1	3,29	RS Components		nts-electroniques
Carte micro SD	27,8	1	27,8	RS Components		m/web/p/carte	Câbles male - male	3,24	1	3,24	RS Components		x/straps-pour-pla
Connecteur Wago	9,57	1	9,57	RS Components		b/p/horniers-c	Total			756,4			

Figure 24 : Tableur des coûts du système Pick To Light

Le système Pick To Light a eu un coût pour l'entreprise, le coût de la partie matérielle s'élève à 756 € comme le montre la figure 24. Celui-ci comprend le matériel qu'il ne sera pas nécessaire d'acheter de nouveau comme les différents microcontrôleurs utilisés, le boîtier 3D, la carte SD, les connecteurs wago, les différents câbles utilisés. En revanche, il comprend également du matériel qu'il faudra peut être commander de nouveau comme les connecteurs JST-SM et les leds s'il faut les remplacer en cas de dysfonctionnement.

Accompagnement du changement

Enfin, la dernière mission de ce stage était d'accompagner l'entreprise pour pérenniser le système déployé. C'est pourquoi j'ai créé un fichier pour les accompagner dans la modification du système et assurer ainsi une passation de maîtrise en interne.

Un avantage de ce système, c'est qu'il a été conçu pour accepter de nouveaux articles que ce soit avec la manière dont le code a été écrit ou la manière dont le système a été développé physiquement. Ce fichier est donc un guide qui leur explique le protocole à suivre pour ajouter de nouveaux articles au système.

Un autre avantage de ce système, c'est qu'en cas de panne, il est possible de le résoudre localement, il n'est pas nécessaire de stopper le fonctionnement du système pour résoudre le problème et celui-ci est résoluble en quelques minutes avec le remplacement de la pièce défectueuse. Le système a donc été pensé pour simplifier la maintenance et le fichier explique la démarche à suivre pour résoudre une panne.

Enfin, j'ai œuvré sur l'accompagnement du changement auprès des opérateurs en commençant par leur présenter le fonctionnement du système pour qu'ils acceptent la venue du système et donc le changement de leur mode opératoire. Je leur ai ensuite expliqué les éléments auxquels il fallait faire attention. Le système a bien été accepté par les opérateurs, ils étaient enthousiastes lors des tests effectués avec eux quant à la venue de ce nouveau système puisqu'ils ont vite compris les enjeux et les bénéfices sur leur quotidien.

J'ai également formé le personnel de l'entreprise pour qu'ils permettent la pérennisation du système, que ce soit concernant la préparation physique des bandeaux de leds, le câblage permettant l'alimentation du système ou la composition du code. La transmission de ces connaissances était essentielle pour que l'entreprise ait la possibilité d'intervenir sur le système.

Un autre intérêt de ce système est qu'il est duplicable, c'est-à-dire que ce système peut être utilisé dans d'autres zones. Mais il est également évolutif, il y a la possibilité d'ajouter des fonctionnalités comme des écrans, des claviers ou bien des boutons. Il a été choisi de commencer par un système utilisant uniquement des leds pour simplifier la première version du système à implanter. Mais un prototype utilisant des écrans (LCD 1602) et des multiplexeurs (PCA9548A), permettant d'afficher la référence de l'article, son emplacement et la quantité à prélever, a été établi et transmis à l'entreprise pour que ceux-ci le mettent en place si besoin à l'avenir.

Conclusion

Pour conclure, pour concevoir, réaliser et implanter un système qui soit modifiable et permettant une maintenance rapide et accessible dans un cadre stricte avec des contraintes, il faut s'inspirer de solutions existantes et les adapter au cadre imposé. Il faut également concevoir une architecture du système qui s'adapte aux contraintes présentes.

Les missions réalisées dans le cadre de ce stage ont été terminées à temps. Dans les faits, l'entièreté des missions qui m'ont été confiées ont été réalisées, le travail restant sera de déployer ce système pour d'autres services ou bien d'améliorer le système en ajoutant des écrans par exemple.

Étant donné que le système a été mis en place récemment, je n'ai pas de retour sur son impact sur les performances de l'entreprise mais les opérateurs ont bien accueilli le système, saluant sa praticité et sa réactivité lors de la préparation d'accessoires.

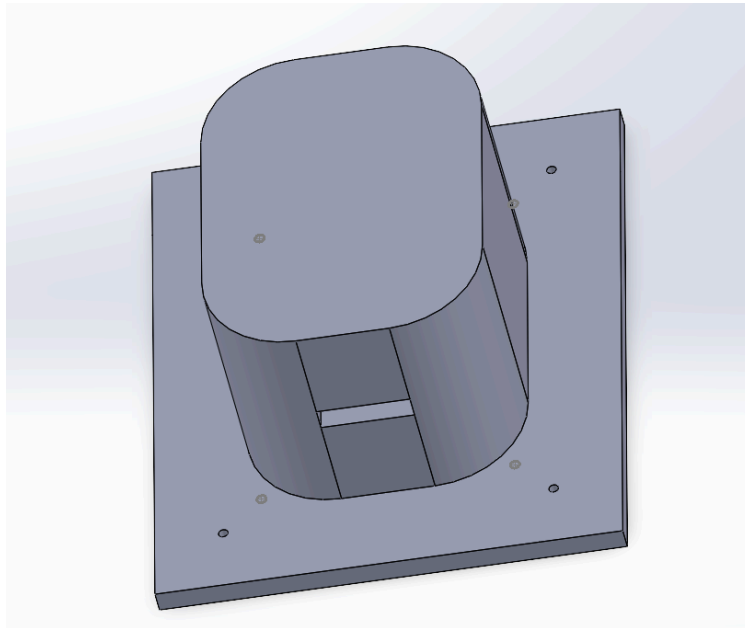
Ce stage a été très formateur que ce soit du côté technique ou du côté organisationnel. Du côté technique, j'ai eu l'occasion de concevoir et de réaliser l'architecture du système, d'implémenter son code et de réaliser son architecture électrique. Du côté organisationnel, j'ai eu l'opportunité de m'organiser pour mener à bien ce projet, j'ai dû collaborer avec les différentes équipes pour réaliser les commandes nécessaires, pour faire dialoguer le système avec Silog et j'ai organisé mon emploi du temps pour respecter le planning adopté, tout en partageant régulièrement l'état d'avancement du projet à l'entreprise et pour avancer sur les tâches nécessaires au bon déroulement de ce projet.

Finalement, j'ai beaucoup progressé en termes de méthodologie, lors de la planification des différentes tâches à effectuer tout au long du stage, en terme d'autonomie, j'ai dû agencer moi-même mon emploi du temps pour réaliser les différentes tâches que je me suis imposé à temps, j'ai également dû faire preuve d'adaptation pour réagir face aux imprévus et aux contraintes imposées.

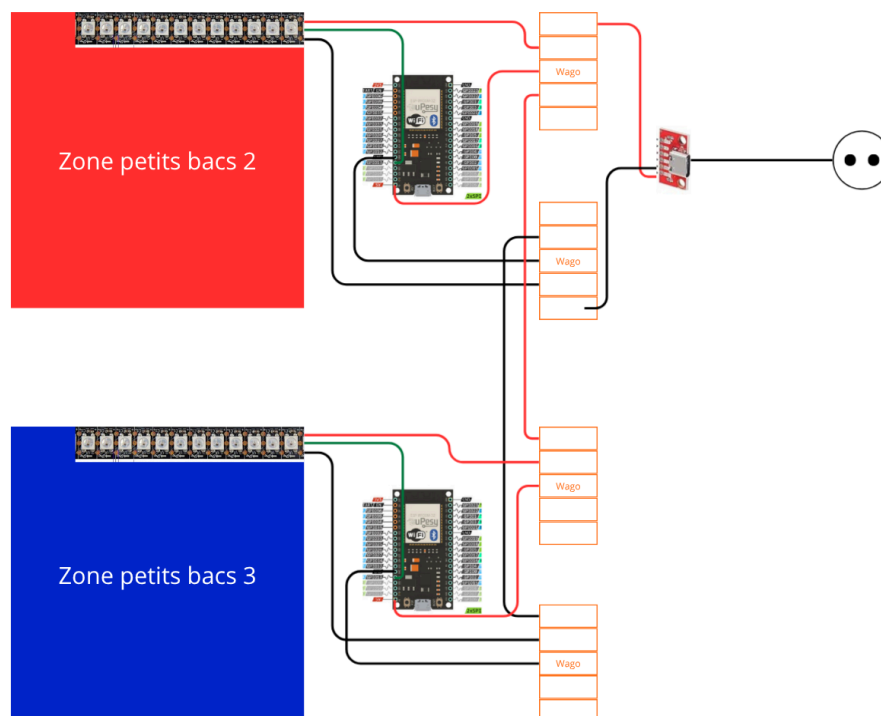
En guise de conclusion, je dirais que cette expérience a été très enrichissante pour moi, me permettant de grandir techniquement et humainement. De plus, elle m'aura permis d'exploiter les connaissances acquises durant mon cursus scolaire dans les secteurs de la programmation, de l'électronique et de l'internet des objets et de les adapter dans l'industrie. Ceci m'a permis d'élargir mes compétences dans ces domaines. J'ai également appris à transmettre mes connaissances sur ces sujets pour accompagner l'entreprise dans le déploiement et l'entretien de ce système. J'espère que ce système aidera l'entreprise à diminuer le nombre de SAV issus de la préparation d'accessoires et qu'il sera possible de déployer ce genre de système dans d'autres zones de travail.

Table des annexes

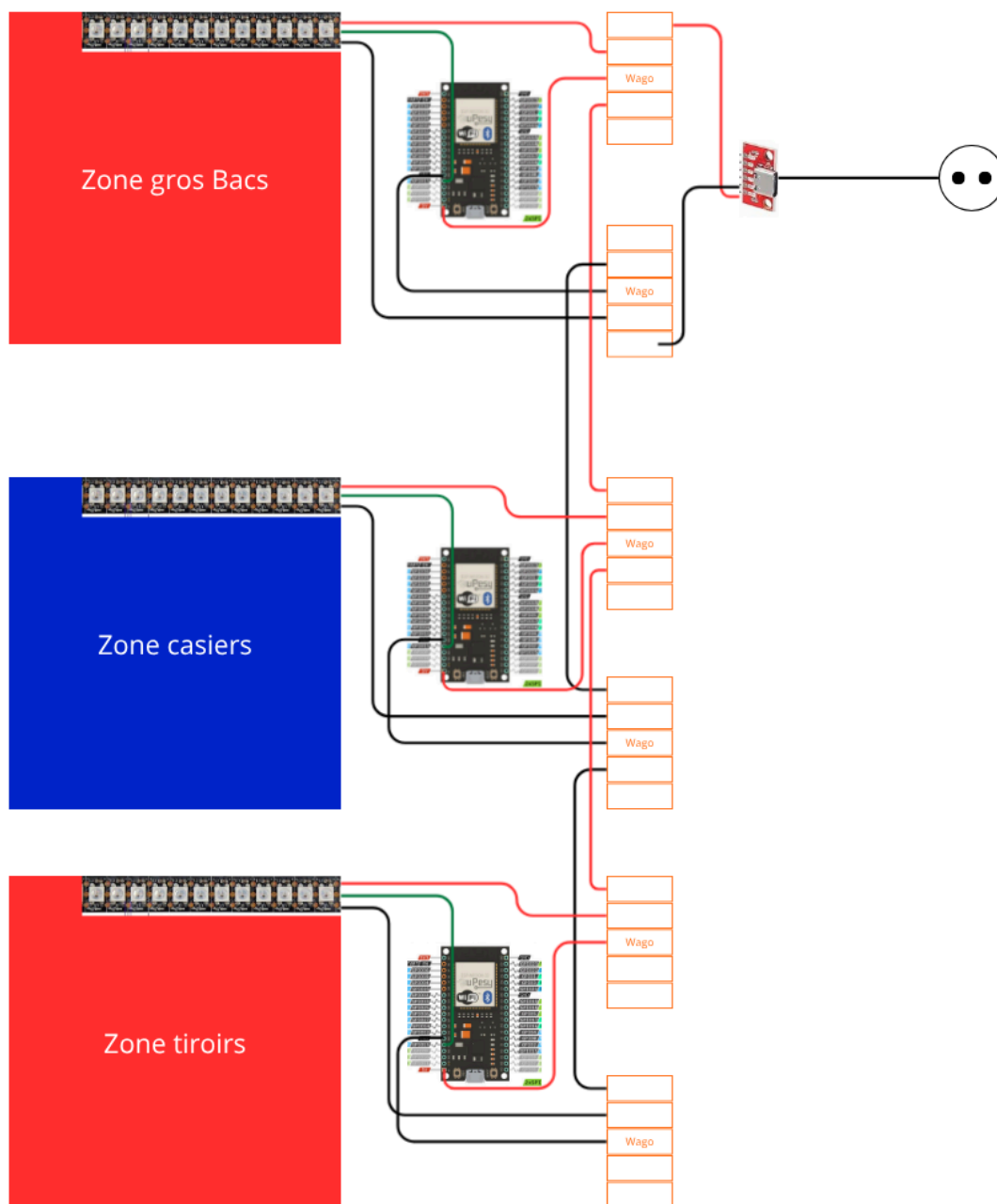
Annexe 1 : Image du fichier 3D du boîtier.....	33
Annexe 2 : Schéma électrique de la deuxième partie droite du système.....	33
Annexe 3 : Schéma électrique de la première partie droite du système.....	34
Annexe 4 : Schéma de la pièce côté gauche.....	35
Annexe 5 : Schéma de la première partie droite de la pièce.....	36
Annexe 6 : Schéma de la deuxième partie droite de la pièce.....	36
Annexe 7 : Schéma de numérotation des articles de la partie gauche de la pièce.....	36
Annexe 8 : Schéma de numérotation des articles de la partie droite de la pièce.....	37
Annexe 9 : Code de la classe Article.....	38
Annexe 10 : Code de la classe Commande.....	39
Annexe 11 : Code de la classe Lecture_bon.....	40
Annexe 12 : Code de la classe Controle_MQTT_Pi.....	43
Annexe 13 : Code du programme principal de la Raspberry Pi 5.....	45
Annexe 14 : Code de la classe Controle_leds.....	46
Annexe 15 : Code de la classe Controle_MQTT.....	47
Annexe 16 : Code du programme principal des ESP32.....	49
Annexe 17 : Code du programme principal des ESP32 avec des écrans.....	51
Annexe 18 : Flux Node-RED.....	51
Annexe 19 : Noeud du texte article sur Node-RED.....	52
Annexe 20 : Noeud de conversion en json sur Node-RED.....	52
Annexe 21 : Noeud MQTT-in sur Node-RED.....	53
Annexe 22 : Noeud de la jauge de quantité sur Node-RED.....	53
Annexe 23 : Noeud du template sur Node-RED.....	54
Annexe 24 : Code du template sur Node-RED.....	62
Annexe 25 : Noeud de la fonction Durée sur Node-RED.....	63
Annexe 26 : Noeud Cadence de Picking sur Node-RED.....	63
Annexe 27 : Noeud MQTT-in 2.....	64
Annexe 28 : Noeud fonction Avancee bon de préparation sur Node-RED.....	64
Annexe 29 : Code Avancee bon de preparation sur Node-RED.....	66
Annexe 30 : Noeud de la jauge Avancée du bon de préparation sur Node-RED.....	67
Annexe 31 : Noeud nombre article restant sur Node-RED.....	67
Annexe 32 : Noeud temps moyen par référence sur Node-RED.....	68
Annexe 33 : Noeud Temps restant bon préparation sur Node-RED.....	68
Annexe 34 : Noeud template 2 sur Node-RED.....	69
Annexe 35 : Code du template 2 sur Node-RED.....	70
Annexe 36 : Noeud Inject 1 sur Node-RED.....	71
Annexe 37 : Noeud fonction Alerte blocage sur Node-RED.....	71
Annexe 38 : Noeud du template 3 sur Node-RED.....	72
Annexe 39 : Code du template 3 sur Node-RED.....	73
Annexe 40 : Noeud inject 2 sur Node-RED.....	74
Annexe 41 : Noeud fonction Etat commande et chrono sur Node-RED.....	75
Annexe 42 : Noeud template 4 sur Node-RED.....	76
Annexe 43 : Code template 4 sur Node-RED.....	77
Annexe 44 : Noeud chronomètre picking sur Node-RED.....	77



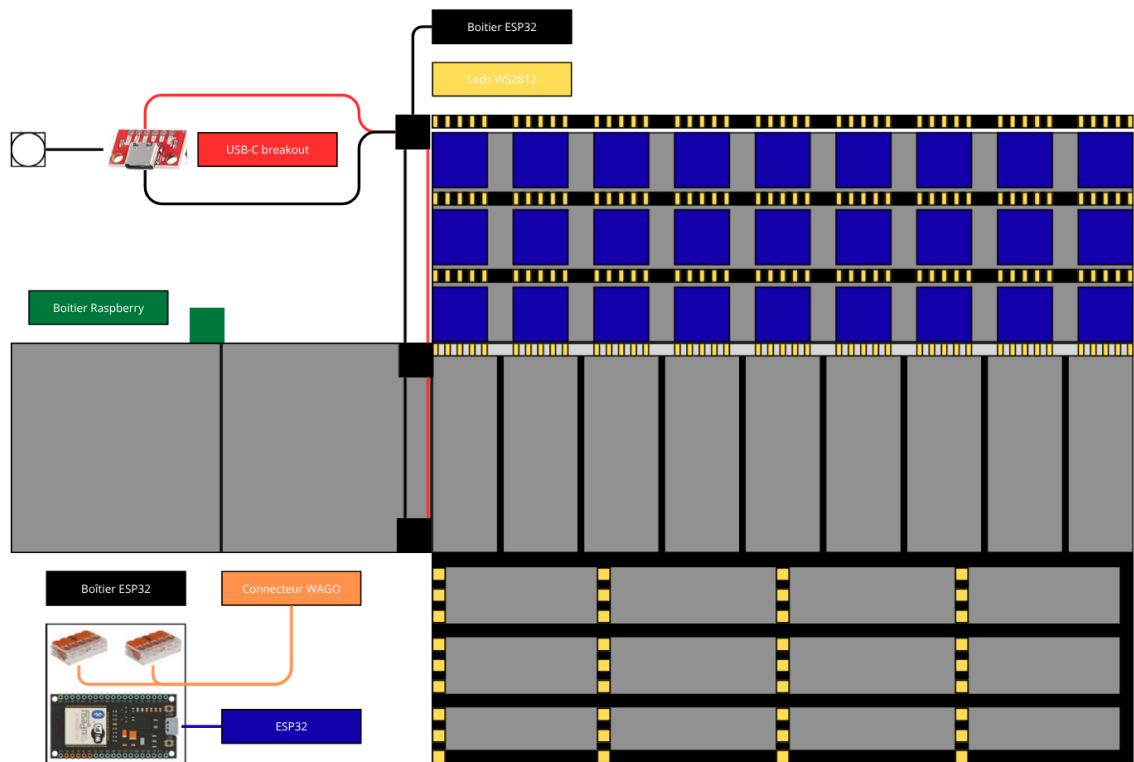
Annexe 1 : Image du fichier 3D du boîtier



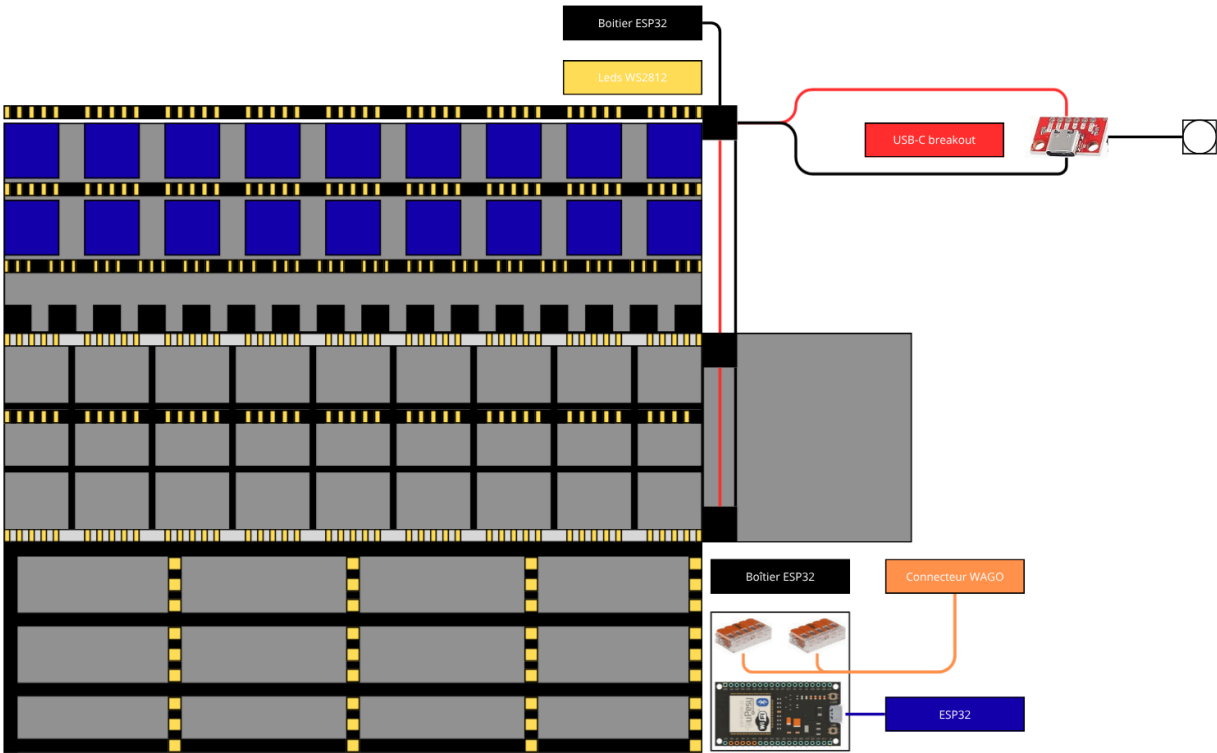
Annexe 2 : Schéma électrique de la deuxième partie droite du système



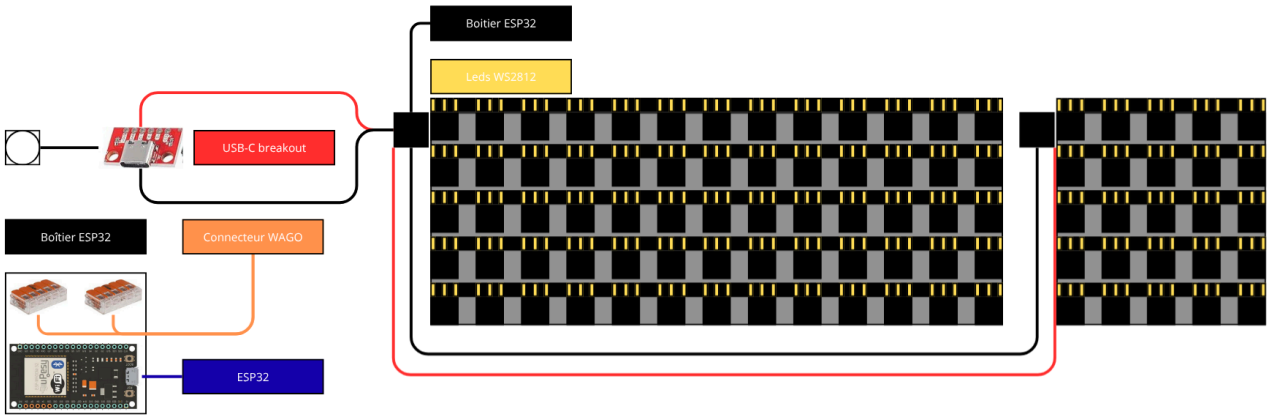
Annexe 3 : Schéma électrique de la première partie droite du système



Annexe 4 : Schéma de la pièce côté gauche



Annexe 5 : Schéma de la première partie droite de la pièce



Annexe 6 : Schéma de la deuxième partie droite de la pièce

Ordinateur		1	2	3	4	5	6	7	8	9	10	11	12	Zone gros bacs
		13	14	15	16	17	18	19	20	21	22	23	24	
		25	26	27	28	29	30	31	32	33	34	35	36	
		101	102	103	104	105	106	107	108	109	110	111	112	Zone casiers
Imprimantes		201			204			207			210			Zone tiroirs
		202			205			208			211			
		203			206			209			212			

Annexe 7 : Schéma de numérotation des articles de la partie gauche de la pièce

Annexe 8 : Schéma de numérotation des articles de la partie droite de la pièce

37

```

    chaine = "La reference : " + self.__reference + ", sa quantite est de : " +
str(self.__quantite) + ", son poids est de : " + str(self.__poids) + " g, et il est dans la case : " +
str(self.__case)
    return chaine

```

Annexe 9 : Code de la classe Article

'''

Commande.py

Cette classe a pour but de définir une commande avec son numéro, sa quantité, et sa liste d'articles.

Elle a pour but de créer un bon de commande.

'''

class Cmd():

#Nous définissons les attributs d'une commande

def __init__(self, numero, quantite, liste_article):

self.__numero = numero

self.__quantite = quantite

self.__liste_article = liste_article

#Nous définissons les accesseurs aux attributs de la classe commande

@property

def quantite(self):

return self.__description

@property

def liste_article(self):

return self.__liste_article

@property

def numero(self):

return self.__numero

'''

Fonction ajouter_article

Elle a pour but d'ajouter un article à une commande.

La fonction ne retourne rien.

La fonction exige un article en entrée.

'''

def ajouter_article(self,article):

self.__liste_article.append(article)

self.__quantite = self.__quantite+1

'''

Fonction enlever_article

Elle a pour but d'enlever un article à une commande.

La fonction ne retourne rien.

La fonction exige en paramètre un article.

'''

```
def enlever_article(self,article):
    self.__liste_article.remove(article)
    self.__quantite = self.__quantite-1
```

'''

Fonction to_string

Elle a pour but de décrire le contenu d'une commande.

La fonction retourne une chaîne de caractère décrivant le contenu d'une commande.

La fonction n'exige aucun paramètre en entrée.

'''

```
def to_string(self):
    chaine = "La commande " + self.__numero + " est compose de :\n"
    for article in self.__liste_article:
        chaine+=article.to_string() +"\n"
    return chaine
```

Annexe 10 : Code de la classe Commande

'''

Lecture_bon.py

Cette classe a pour but de définir la lecture d'un bon de commande envoyé par l'ERP.

Elle a pour but d'analyser le contenu d'une commande envoyé par l'ERP.

'''

```
import re
from Commande import Cmd
from Article import Art
```

```
class Lecture():
    #Nous définissons les attributs d'un bon de commande
    def __init__(self,nom):
        self.__nom = nom
        fichier = open(nom,"r")
        self.__contenu = fichier.readlines()
        fichier.close()
```

'''

Fonction lire_bon_commande

Elle a pour but de lire le contenu d'un fichier associé à un bon de commande et de transmettre la commande.

La fonction retourne une commande.

La fonction n'exige aucun paramètre en entrée.

'''

```
def lire_bon_commande(self,numero):
    commande = Cmd(numero,0,[])
    for ligne in self.__contenu :
        ligne = ligne.strip()
        lecture = re.split(";", ligne)
        reference = lecture[0]
        poids = lecture[1]
        poids = float(poids.replace(",","."))
        case = int(lecture[2])
        quantite = int(lecture[3])
        nouvel_article = Art(reference,quantite,poids,case)
        commande.ajouter_article(nouvel_article)
    return commande
```

Annexe 11 : Code de la classe Lecture_bon

'''

Controle_MQTT_Pi.py

Cette classe a pour but de définir le protocole MQTT utilisé par le raspberry pi.
Elle a pour but de permettre la communication entre le raspberry pi, l'ESP32 et le
tableau de bord Node-RED pour permettre son actualisation en temps réel.

'''

```
import paho.mqtt.client as mqtt
import json
import time
import threading
```

```
class GestionnaireMQTT:
```

```
    #Nous définissons les attributs du protocole MQTT
```

```
    def __init__(self, broker="localhost", port=1883):
```

```
        self.broker = broker
```

```
        self.port = port
```

```
        self.topic_cmd = "systeme/ptl/article"
```

```
        self.topic_esp1 = "systeme/ptl/esp1/article"
```

```
        self.topic_esp2 = "systeme/ptl/esp2/article"
```

```
        self.topic_esp3 = "systeme/ptl/esp3/article"
```

```
        self.topic_esp4 = "systeme/ptl/esp4/article"
```

```
        self.topic_esp5 = "systeme/ptl/esp5/article"
```

```
self.topic_esp6 = "systeme/ptl/esp6/article"
self.topic_esp7 = "systeme/ptl/esp7/article"
self.topic_esp8 = "systeme/ptl/esp8/article"
self.topic_esp10 = "systeme/ptl/esp10/article"
self.topic_conf = "systeme/ptl/+/confirmation"
```

```
self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
'''
```

Fonction connecter

Elle a pour but de s'abonner au topic sur lequel l'ESP32 publie.

La fonction ne retourne rien.

La fonction n'exige aucun paramètre en entrée.

'''

def connecter(self):

```
    try:
        self.client.connect(self.broker, self.port, 60)
        self.client.loop_start()
        # On s'abonne au topic de confirmation venant de l'ESP32
        self.client.subscribe(self.topic_conf)
        print(f"MQTT : Connecté et abonné à {self.topic_conf}")
    except Exception as e:
        print(f"MQTT : Échec de connexion : {e}")
'''
```

Fonction envoyer_etat

Elle a pour but d'envoyer l'état d'un article au tableau de bord avec son emplacement, sa référence, sa quantité et son statut.

La fonction ne retourne rien.

La fonction exige en paramètre, l'emplacement, la référence, la quantité et le statut d'un article.

'''

def envoyer_etat(self, case, ref, qty, status):

```
    payload = {
        "case": case,
        "ref": ref,
        "qty": qty,
        "status": status
    }
    self.client.publish(self.topic_cmd, json.dumps(payload))
'''
```

Fonction envoyer_liste_partitionnee

Elle a pour but d'envoyer un bon de commande complet aux différents ESP32.

La fonction ne retourne rien.

La fonction exige en paramètre une commande.

'''

```
def envoyer_liste_partitionnee(self, commande):
```

```
    # Séparation des articles
```

```
    articles_esp1 = [a for a in commande.liste_article if 1 <= a.case <= 100]
```

```
    articles_esp2 = [a for a in commande.liste_article if 101 <= a.case <= 200]
```

```
    articles_esp3 = [a for a in commande.liste_article if 201 <= a.case <= 300]
```

```
    articles_esp4 = [a for a in commande.liste_article if 301 <= a.case <= 400]
```

```
    articles_esp5 = [a for a in commande.liste_article if 401 <= a.case <= 500]
```

```
    articles_esp6 = [a for a in commande.liste_article if 501 <= a.case <= 600]
```

```
    articles_esp7 = [a for a in commande.liste_article if 601 <= a.case <= 700]
```

```
    articles_esp8 = [a for a in commande.liste_article if 701 <= a.case <= 800]
```

```
    articles_esp10 = [a for a in commande.liste_article if 901 <= a.case <= 1000]
```

```
def preparer_payload(articles):
```

```
    # On crée le dictionnaire
```

```
    data = {
```

```
        "status": "nouvelle_commande",
```

```
        "articles": [{"case": a.case, "ref": a.reference, "qty": a.quantite} for a in  
articles],
```

```
        "numero_commande": commande.numero
```

```
    }
```

```
    # CRITIQUE : Convertir le dictionnaire en chaîne JSON
```

```
    return json.dumps(data)
```

```
payload1 = preparer_payload(articles_esp1)
```

```
self.client.publish(self.topic_esp1, payload1)
```

```
print(f"MQTT : {len(articles_esp1)} articles envoyés à l'ESP 1.")
```

```
time.sleep(0.1)
```

```
payload2 = preparer_payload(articles_esp2)
```

```
self.client.publish(self.topic_esp2, payload2)
```

```
print(f"MQTT : {len(articles_esp2)} articles envoyés à l'ESP 2.")
```

```
time.sleep(0.1)
```

```
payload3 = preparer_payload(articles_esp3)
```

```
self.client.publish(self.topic_esp3, payload3)
```

```
print(f"MQTT : {len(articles_esp3)} articles envoyés à l'ESP 3.")
```

```
time.sleep(0.1)
```

```
payload4 = preparer_payload(articles_esp4)
```

```
self.client.publish(self.topic_esp4, payload4)
```

```
print(f"MQTT : {len(articles_esp4)} articles envoyés à l'ESP 4.")
time.sleep(0.1)
payload5 = preparer_payload(articles_esp5)
self.client.publish(self.topic_esp5, payload5)
print(f"MQTT : {len(articles_esp5)} articles envoyés à l'ESP 5.")
time.sleep(0.1)
payload6 = preparer_payload(articles_esp6)
self.client.publish(self.topic_esp6, payload6)
print(f"MQTT : {len(articles_esp6)} articles envoyés à l'ESP 6.")
time.sleep(0.1)
payload7 = preparer_payload(articles_esp7)
self.client.publish(self.topic_esp7, payload7)
print(f"MQTT : {len(articles_esp7)} articles envoyés à l'ESP 7.")
time.sleep(0.1)
payload8 = preparer_payload(articles_esp8)
self.client.publish(self.topic_esp8, payload8)
print(f"MQTT : {len(articles_esp8)} articles envoyés à l'ESP 8.")
time.sleep(0.1)
payload10 = preparer_payload(articles_esp10)
self.client.publish(self.topic_esp10, payload10)
print(f"MQTT : {len(articles_esp8)} articles envoyés à l'ESP 10.")
time.sleep(0.1)
'''
```

Fonction deconnecter

Elle a pour but de remettre tout à zéro et de couper la connection au broker.

La fonction ne retourne rien.

La fonction n'exige aucun paramètre en entrée.

'''

```
def deconnecter(self):
    self.client.loop_stop()
    self.client.disconnect()
    print("MQTT : Déconnecté")
```

Annexe 12 : Code de la classe Controle_MQTT_PI

'''

PTL_avec_ESP32_PI.py

Ce programme envoie de données vers un ESP32 qui va piloter un bandeau de leds, un écran et un bouton de validation.

Il a pour but de tester l'envoi de données vers un ESP32.

L'arrêt sera provoqué par l'arrêt du programme ou par la pression du bouton d'arrêt.

Il n'y a aucun paramètre en entrée.

'''

```
from datetime import datetime
from pathlib import Path
from Lecture_bon import Lecture
from Controle_MQTT_PI import GestionnaireMQTT
import threading
import json
import time
```

```
numero_commande = 1
```

'''

Fonction envoi_commande

Elle permet d'envoyer la totalité d'un bon de commande à l'ESP32.

Elle attend le retour de l'ESP32 pour envoyer le deuxième bon de commande.

Elle exige en paramètres, un gestionnaire_MQTT.

Elle ne retourne rien.

'''

```
def envoi_commande(mqtt_hub):
    print("Nettoyage de l'ancien affichage du Dashboard")
    mqtt_hub.envoyer_etat(0,"",0,"repos")
    time.sleep(0.1)
    commande = contenu_commande.lire_bon_commande(numero_commande)
    mqtt_hub.envoyer_liste_partitionnee(commande)
    print(f"Commande envoyée. En attente des opérateurs...")
    for article in commande.liste_article :
        #Envoie données à Node-RED
        mqtt_hub.envoyer_etat(article.case, article.reference, article.quantite, "en_cours")
    print("Données synchronisées avec Node-RED.")
```

```
if __name__ == "__main__":
    com = GestionnaireMQTT()
    com.connecter()
    contenu_commande = Lecture("/home/matys/Téléchargements/AR000002.txt")
    chemin_fichier = Path(r"/home/matys/Téléchargements/AR000002.txt")
    #/home/matys/Téléchargements/AR000002.txt
    #/mnt/PTL/ListeAccessoireLED.txt
```

try :

Benoist Matys, Stage en IoT
2025-2026

```
print("Connecté au Broker MQTT et matériel prêt.")
date_avant = ""
while True :
    try :
        timestamp = chemin_fichier.stat().st_mtime
        date_modification = datetime.fromtimestamp(timestamp)
        if(date_modification != date_avant):
            print("le fichier a été modifié")
            contenu_commande = Lecture("/home/matys/Téléchargements/AR000002.txt")
            envoi_commande(com)
            date_avant = date_modification
    except FileNotFoundError:
        print("Désolé, le fichier n'existe pas")

    time.sleep(1)

except (KeyboardInterrupt, SystemExit):
    print("\nArrêt du programme détecté...")

finally:
    print("Nettoyage du matériel en cours...")
    com.deconnecter()
    print("Système éteint proprement.")
```

Annexe 13 : Code du programme principal de la Raspberry Pi 5

'''

Controle_leds.py

Cette classe a pour but de définir un bandeau de leds avec l'ESP32 en microPython.
Elle a pour but d'indiquer les articles à prélever avec l'allumage de leds.

'''

import neopixel

from machine import Pin

import time

class Bandeau_led:

#Nous définissons les attributs d'un bandeau de leds

def __init__(self, led_count, pin_data):

self.nb_led = led_count

self.np = neopixel.NeoPixel(Pin(pin_data), led_count)

self.COLORS = {

"GROSSE": (255, 255, 0),

"MOYENNE": (232, 62, 1),

"PETITE": (0, 0, 255),

"VALID": (0, 255, 0),

```
        "OFF": (0, 0, 0)
    }
'''
```

Fonction allumer_led

Elle a pour but d'allumer une led précise d'une certaine couleur en fonction de la quantité à prélever.

La fonction ne retourne rien.

La fonction exige en paramètre, l'indice de la led à allumer et la quantité à prélever.

```
'''
def allumer_bandeau(self, emplacement, quantite):
    color = self.COLORS["VALID"]
    for i in range(5):
        self.np[5*emplacement + i] = color
    self.np.write()
'''
```

Fonction eteindre_led

Elle a pour but d'eteindre une led précise.

La fonction ne retourne rien.

La fonction exige en paramètre, l'indice de la led à éteindre.

```
'''
def eteindre_toutes(self):
    for i in range(self.nb_led):
        self.np[i] = self.COLORS["OFF"]
    self.np.write()
'''
```

Fonction valider_led

Elle a pour but d'allumer en vert une led puis de l'éteindre après 0.5s pour valider la prise d'un article.

La fonction ne retourne rien.

La fonction exige en paramètre, l'indice de la led de l'article à valider.

```
'''
def valider_bandeau(self, emplacement):
    for i in range(5):
        self.np[5*emplacement + i] = self.COLORS["VALID"]
    self.np.write()
    time.sleep(0.5)
    for i in range(5):
        self.np[5*emplacement + i] = self.COLORS["OFF"]
    self.np.write()
'''
```

Annexe 14 : Code de la classe Controle_leds

'''

Controle_MQTT.py

Cette classe a pour but de définir un protocole MQTT avec l'Esp32 en microPython.
Elle a pour but de permettre la communication entre le raspberry pi et le tableau de bord Node-RED pour permettre son actualisation en temps réel.

'''

from umqtt.simple import MQTTClient

import ujson

class GestionnaireMQTT:

 #Nous définissons les attributs d'un protocole MQTT

 def __init__(self, id_esp, broker="10.42.0.1", port=1883):

 self.id_esp = id_esp

 self.broker = broker

 self.port = port

 self.topic_cmd = f"systeme/ptl/{id_esp}/article"

 self.topic_conf = f"systeme/ptl/{id_esp}/confirmation"

 self.client = MQTTClient(f"ESP32_{id_esp}", self.broker, port=self.port)

'''

Fonction connecter

Elle a pour but de connecter le raspberry pi au broker.

La fonction ne retourne rien.

La fonction exige en paramètre une fonction callback.

'''

def connecter(self, callback_func):

 self.client.set_callback(callback_func)

 try:

 self.client.connect()

 self.client.subscribe(self.topic_cmd)

 print("MQTT ESP32 : Connecté au broker")

 except Exception as e:

 print("Erreur connexion MQTT:", e)

'''

Fonction verifier_messages

Elle a pour but de vérifier si un message venant du raspberry pi est arrivé.

La fonction ne retourne rien.

La fonction n'exige aucun paramètre en entrée.

'''

def verifier_messages(self):

 self.client.check_msg()

Annexe 15 : Code de la classe Controle_MQTT

'''

main.py

Ce programme traite la réception des données du raspberry pi par l'ESP32.

Il a pour but de tester la réception de données du raspberry pi.

L'arrêt sera provoqué par l'arrêt du programme.

Il n'y a aucun paramètre en entrée.

'''

import ujson

import time

import network

from machine import Pin

from umqtt.simple import MQTTClient

from Controle_MQTT import GestionnaireMQTT

from Controle_leds import Bandeau_led

'''

Fonction connexion_wifi_ouvert

Elle permet à l'ESP32 de se connecter au réseau.

Elle exige en paramètres le nom du wifi sur lequel il doit se connecter.

Elle ne retourne rien.

'''

def connexion_wifi_ouvert(ssid):

 wlan = network.WLAN(network.STA_IF)

 wlan.active(True)

 if not wlan.isconnected():

 print(f"Connexion au réseau ouvert : {ssid}...")

 wlan.connect(ssid, "")

 tentatives = 0

 while not wlan.isconnected() and tentatives < 15:

 print(".")

 time.sleep(1)

 tentatives += 1

 if wlan.isconnected():

 print('Connecté !')

 print('Configuration réseau (IP/Masque/GW/DNS) :', wlan.ifconfig())

 else:

 print('Erreur : Impossible de se connecter au Hotspot.')

connexion_wifi_ouvert('CoulidoorPTLPI')

bandeau = Bandeau_led(500, 13)

dernier_ordre = None

'''

Fonction callback_mqtt

Benoist Matys, Stage en IoT

2025-2026

Elle permet de traiter les données envoyées par le raspberry pi.
Elle exige en paramètres le topic sur lesquels les données sont publiées et le message envoyé.

Elle ne retourne rien.

'''

```
def callback_mqtt(topic, msg):
    global dernier_ordre
    try:
        data = ujson.loads(msg)
        dernier_ordre = data
    except:
        print("Erreur JSON")
```

```
com = GestionnaireMQTT(id_esp="esp1",broker="10.42.0.1")
com.connecter(callback_mqtt)
print("ESP32 Pret !")
time.sleep(3)
```

```
while True:
    com.verifier_messages()
    if dernier_ordre and dernier_ordre.get("status")== "nouvelle_commande":
        articles = dernier_ordre.get("articles")
        print("Allumage du chemin de picking...")
        bandeau.eteindre_toutes()
        for article in articles:
            bandeau.allumer_bandeau(article['case']-1, article['qty'])
        for i, article in enumerate(articles):
            print(f"Article {i+1}/{len(articles)} : {article['ref']}")
        dernier_ordre = None
        time.sleep(0.1)
```

Annexe 16 : Code du programme principal des ESP32

'''

main.py

Ce programme traite la réception des données du raspberry pi par l'ESP32.
Il a pour but de tester la réception de données du raspberry pi.
L'arrêt sera provoqué par l'arrêt du programme.
Il n'y a aucun paramètre en entrée.

'''

```
import ujson
import time
import network
from machine import Pin
```

Benoist Matys, Stage en IoT
2025-2026

```
from umqtt.simple import MQTTClient
from Controle_MQTT import GestionnaireMQTT
from Controle_LCD import Ecran
from esp32_i2c_lcd import I2cLcd
"""
```

Fonction connexion_wifi_ouvert

Elle permet à l'ESP32 de se connecter au réseau.

Elle exige en paramètres le nom du wifi sur lequel il doit se connecter.

Elle ne retourne rien."""

```
def connexion_wifi_ouvert(ssid):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print(f"Connexion au réseau ouvert : {ssid}...")
        wlan.connect(ssid, "")
        tentatives = 0
        while not wlan.isconnected() and tentatives < 15:
            print(".")
            time.sleep(1)
            tentatives += 1

    if wlan.isconnected():
        print('Connecté !')
        print('Configuration réseau (IP/Masque/GW/DNS) :', wlan.ifconfig())
    else:
        print('Erreur : Impossible de se connecter au Hotspot.')
```

connexion_wifi_ouvert('CoulidoorPTLPI')

```
ecran = Ecran(3)
dernier_ordre = None
"""
```

Fonction callback_mqtt

Elle permet de traiter les données envoyées par le raspberry pi.

Elle exige en paramètres le topic sur lesquels les données sont publiées et le message envoyé.

Elle ne retourne rien.

"""

```
def callback_mqtt(topic, msg):
    global dernier_ordre
    try:
        data = ujson.loads(msg)
        dernier_ordre = data
    except:
```

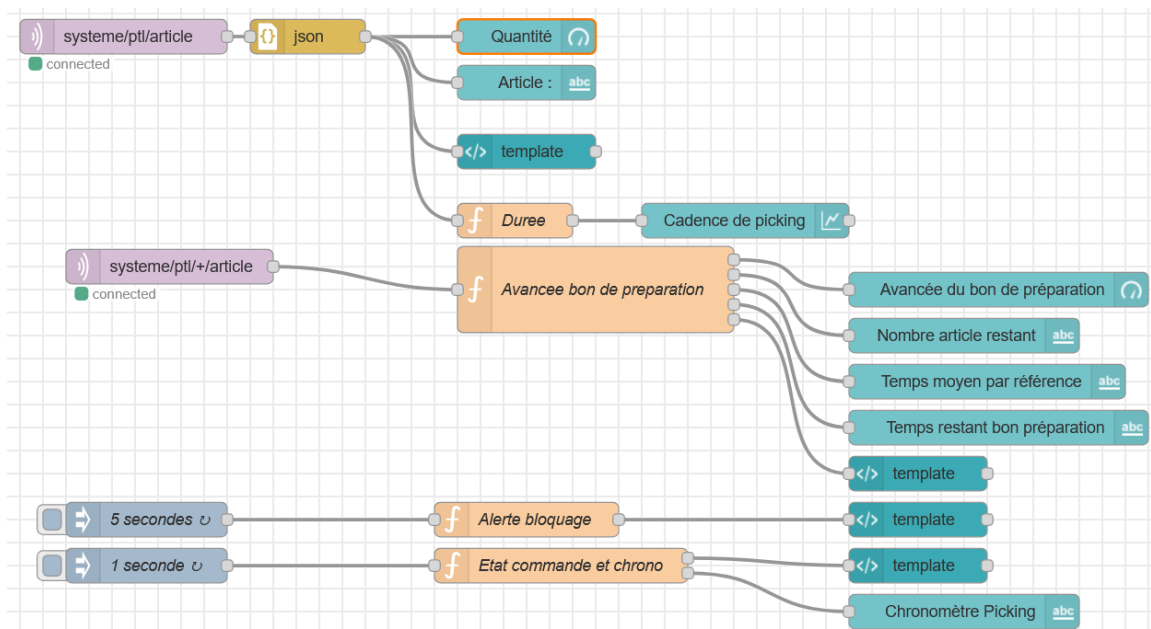
Benoist Matys, Stage en IoT
2025-2026

```
print("Erreur JSON")

com = GestionnaireMQTT(id_esp="esp10",broker="10.42.0.1")
com.connecter(callback_mqtt)
print("ESP32 Pret !")
time.sleep(3)

while True:
    com.verifier_messages()
    if dernier_ordre and dernier_ordre.get("status")== "nouvelle_commande":
        articles = dernier_ordre.get("articles")
        print("Allumage du chemin de picking...")
        ecran.effacer()
        for article in articles:
            ecran.afficher_article(article['case']-901, article['ref'], article['case'], article['qty'])
        for i, article in enumerate(articles):
            print(f"Article {i+1}/{len(articles)} : {article['ref']}")
        dernier_ordre = None
        time.sleep(0.1)
```

Annexe 17 : Code du programme principal des ESP32 avec des écrans



Annexe 18 : Flux Node-RED

Edit text node

Delete Cancel Done

Properties

Group [Picking] Suivi

Size auto

Label Article :

Value format {{msg.payload.ref}} - {{msg.payload.status}}

Layout

label value label value label value

label value label value

Style ☐ Apply Style

Class Optional CSS class name(s) for widget

Name

Annexe 19 : Noeud du texte article sur Node-RED

Edit json node

Delete Cancel Done

Properties

Action Always convert to JavaScript Object

Property msg. payload

Name Name

Annexe 20 : Noeud de conversion en json sur Node-RED

Edit mqtt in node

Delete Cancel Done

Properties

Server localhost:1883

Action Subscribe to single topic

Topic systeme/ptl/article

QoS 2

Output auto-detect (parsed JSON object, string or buffer)

Name Name

Annexe 21 : Noeud MQTT-in sur Node-RED

Edit gauge node

Delete Cancel Done

Properties

Group [Picking] Suivi

Size auto

Type Gauge

Label Quantité

Value format {{payload.qty}}

Units optional sub-label

Range min 0 max 20

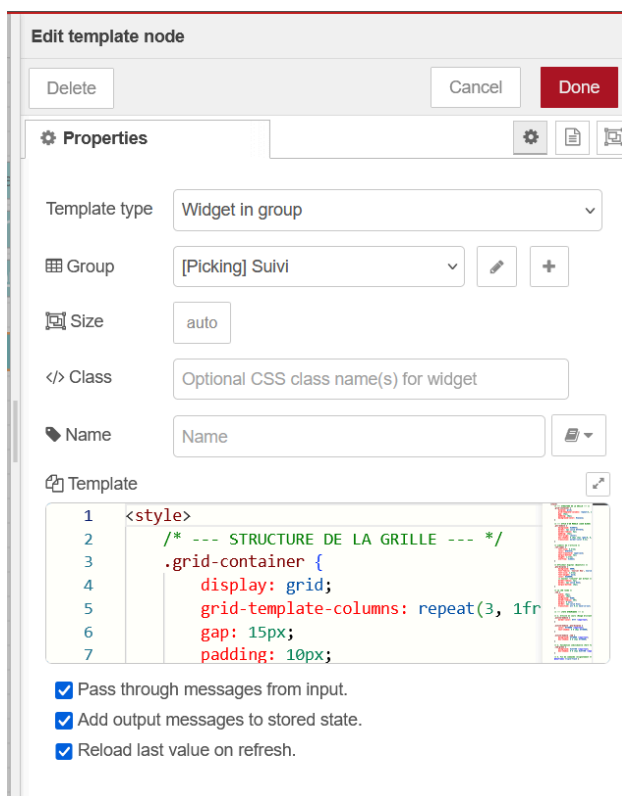
Colour gradient

Sectors 0 ... 5 ... 10 ... 20

Fill gauge from centre. ☐

Class Optional CSS class name(s) for widget

Annexe 22 : Noeud de la jauge de quantité sur Node-RED



Annexe 23 : Noeud du template sur Node-RED

```
<style>
  .zones-container {
    display: flex;
    flex-direction: column;
    gap: 30px;
    padding: 10px;
    background-color: #111;
  }

  .zone-section {
    background-color: #1a1a1a;
    border: 1px solid #333;
    border-radius: 8px;
    padding: 15px;
  }

  .zone-title {
    color: #fff;
    font-family: sans-serif;
    font-size: 1.4rem;
  }
```

```
    font-weight: bold;
    margin-bottom: 15px;
    border-bottom: 2px solid #1808f3;
    padding-bottom: 5px;
}

.grid-container {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(110px, 1fr));
    gap: 12px;
}

.ptl-module {
    background: #1808f3;
    border: 3px solid #f7f4f4;
    border-radius: 8px;
    padding: 10px 5px;
    text-align: center;
    box-shadow: 0 5px 10px rgba(0, 0, 0, 0.4);
    transition: border-color 0.3s;
}

.ref-label {
    font-size: 0.75rem;
    color: #f8f5f5;
    text-transform: uppercase;
    margin-bottom: 5px;
    height: 1.2rem;
    overflow: hidden;
    white-space: nowrap;
    text-overflow: ellipsis;
}

.qty-display {
    background: #000;
    font-family: 'Courier New', Courier, monospace;
    font-size: 1.8rem;
    font-weight: bold;
    color: #2b0000;
    border-radius: 4px;
    border: 2px solid #222;
}
```

```

        margin-bottom: 8px;
    }

    .led {
        width: 14px;
        height: 14px;
        background: #400;
        border-radius: 50%;
        margin: 0 auto;
        border: 2px solid #111;
        transition: all 0.2s ease-in-out;
    }

    .active-module {
        border-color: #fff !important;
    }

    .active-module .qty-display {
        color: #ff0000 !important;
        text-shadow: 0 0 10px #ff0000;
    }

    .active-module .led {
        background: #ff0000 !important;
        box-shadow: 0 0 15px #ff0000;
    }

    .led-green {
        background: #15ff00 !important;
        box-shadow: 0 0 15px #15ff00 !important;
    }

    @keyframes triple-flash {

        0%,
        16%,
        33%,
        50%,
        66%,
        83% {
            background: #15ff00;
        }
    }

```

```

        box-shadow: 0 0 20px #15ff00;
    }

    17%,
    32%,
    51%,
    65%,
    84%,
    100% {
        background: #400;
        box-shadow: none;
    }
}

.led-finish {
    animation: triple-flash 3s ease-in-out forwards;
}
</style>

<div class="zones-container">

    <div class="zone-section">
        <div class="zone-title">Zone 1 (Cases 1 - 36)</div>
        <div class="grid-container">
            <div ng-repeat="i in getPlage(1, 36)" class="ptl-module"
                ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
                <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
                <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
                <div class="led"
                    ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
                </div>
            </div>
        </div>
    </div>

    <div class="zone-section">
        <div class="zone-title">Zone 2 (Cases 101 - 112)</div>

```

```

<div class="grid-container">
  <div ng-repeat="i in getPlage(101, 112)" class="ptl-module"
    ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
    <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
    <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
    <div class="led"
      ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
    </div>
  </div>
</div>

<div class="zone-section">
  <div class="zone-title">Zone 3 (Cases 201 - 212)</div>
  <div class="grid-container">
    <div ng-repeat="i in getPlage(201, 212)" class="ptl-module"
      ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
      <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
      <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
      <div class="led"
        ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
      </div>
    </div>
  </div>
</div>

<div class="zone-section">
  <div class="zone-title">Zone 4 (Cases 301 - 318)</div>
  <div class="grid-container">
    <div ng-repeat="i in getPlage(301, 318)" class="ptl-module"
      ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
      <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref

```

```
: 'Emp ' + i }}</div>
    <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
    <div class="led"
        ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
    </div>
</div>
</div>
</div>

<div class="zone-section">
    <div class="zone-title">Zone 5 (Cases 401 - 427)</div>
    <div class="grid-container">
        <div ng-repeat="i in getPlage(401, 427)" class="ptl-module"
            ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
            <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
            <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
            <div class="led"
                ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
            </div>
        </div>
    </div>
</div>

<div class="zone-section">
    <div class="zone-title">Zone 6 (Cases 501 - 512)</div>
    <div class="grid-container">
        <div ng-repeat="i in getPlage(501, 512)" class="ptl-module"
            ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
            <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
            <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
            <div class="led"
                ng-class="{ 'led-green': etats[i].status == 'valide',
```

```
'led-finish': globalStatus == 'termine_tout'}">
    </div>
</div>
</div>
</div>

<div class="zone-section">
    <div class="zone-title">Zone 7 (Cases 601 - 681)</div>
    <div class="grid-container">
        <div ng-repeat="i in getPlage(601, 681)" class="ptl-module"
            ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
            <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
            <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
            <div class="led"
                ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
            </div>
        </div>
    </div>
</div>
</div>

<div class="zone-section">
    <div class="zone-title">Zone 8 (Cases 701 - 725)</div>
    <div class="grid-container">
        <div ng-repeat="i in getPlage(701, 725)" class="ptl-module"
            ng-class="{ 'active-module': etats[i].status ==
'en_cours' }">
            <div class="ref-label">{{ (etats[i].ref) ? etats[i].ref
: 'Emp ' + i }}</div>
            <div class="qty-display">{{ (etats[i].status ==
'en_cours') ? etats[i].qty : '--' }}</div>
            <div class="led"
                ng-class="{ 'led-green': etats[i].status == 'valide',
'led-finish': globalStatus == 'termine_tout' }">
            </div>
        </div>
    </div>
</div>
</div>
</div>
```

</div>

<script>

```
(function(scope) {
  scope.etats = {};
  scope.globalStatus = 'repos';

  scope.getPlage = function(min, max) {
    var t = [];
    for (var i = min; i <= max; i++) {
      t.push(i);
    }
    return t;
  };

  scope.$watch('msg.payload', function(msg) {
    if (!msg) return;

    if (msg.status === 'termine_tout') {
      scope.globalStatus = 'termine_tout';

      const plages = [
        {deb: 1, fin: 36}, {deb: 101, fin: 112}, {deb: 201, fin: 212}, {deb: 301, fin: 318},
        {deb: 401, fin: 427}, {deb: 501, fin: 512}, {deb: 601, fin: 681}, {deb: 701, fin: 725}
      ];

      plages.forEach(plage => {
        for (let i = plage.deb; i <= plage.fin; i++) {
          if (scope.etats[i]) {
            scope.etats[i].status = 'repos';
            scope.etats[i].ref = 'Emp ' + i;
          }
        }
      });

      setTimeout(function() {
        scope.etats = {};
        scope.globalStatus = 'repos';
      }, 1000);
    }
  });
});
```

```

        scope.$apply();
    }, 3500);
}

else if (msg.status === 'repos') {
    scope.etats = {};
    scope.globalStatus = 'repos';
}

else if (msg.case) {
    let id = parseInt(msg.case);

    if (msg.status === 'en_cours') {
        scope.etats[id] = { ref: msg.ref, qty: msg.qty, status:
'en_cours' };
    }

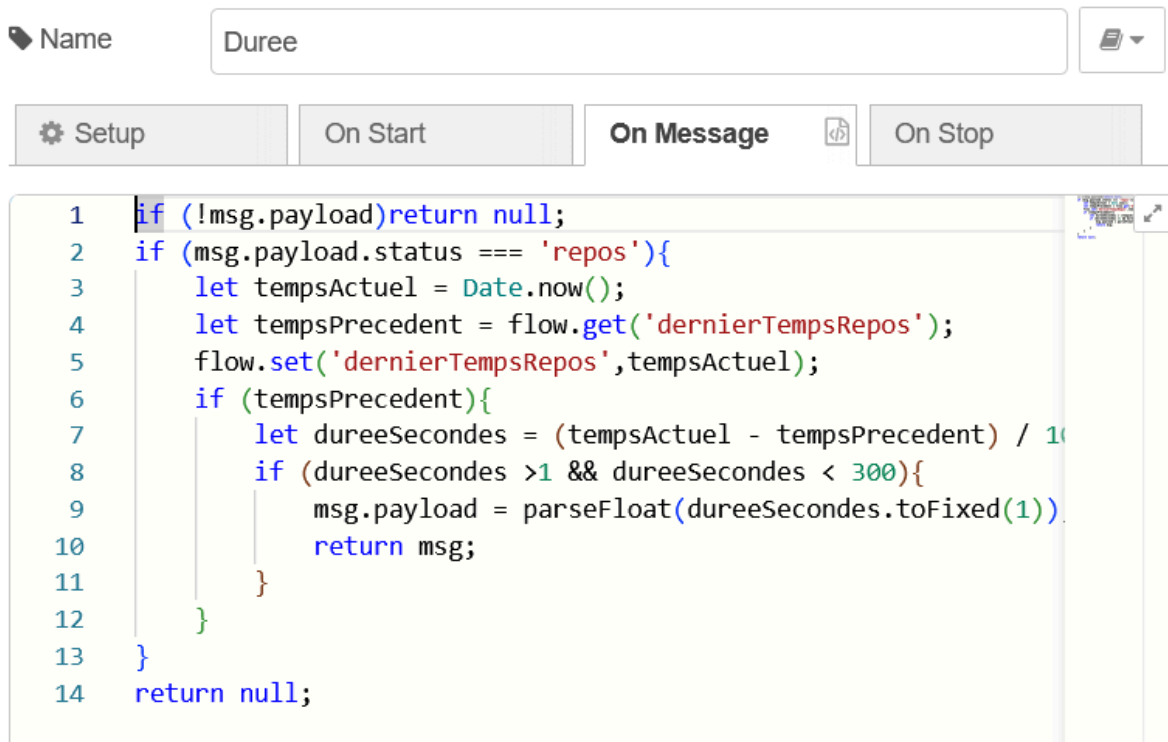
    if (msg.status === 'valide') {
        if(scope.etats[id]) {
            scope.etats[id].status = 'valide';
            scope.etats[id].qty = '--';
        }

        setTimeout(function() {
            if (scope.globalStatus !== 'termine_tout') {
                scope.etats[id] = {
                    ref: 'Emp ' + id,
                    qty: '--',
                    status: 'repos'
                };
                scope.$apply();
            }
        }, 500);
    }
}

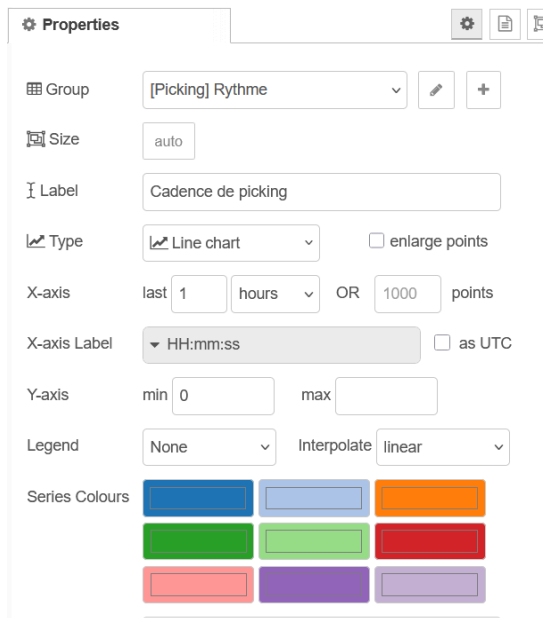
});
})(scope);
</script>

```

Annexe 24 : Code du template sur Node-RED



Annexe 25 : Noeud de la fonction Durée sur Node-RED



Annexe 26 : Noeud Cadence de Picking sur Node-RED

⚙️

Properties

⚙️

📄

🖨️

🌐

Server

localhost:1883

✎

+

👉

Action

Subscribe to single topic

▼

📋

Topic

systeme/ptl/+/article

🌟

QoS

0

▼

👉

Output

a parsed JSON object

▼

🏷️

Name

Name

Annexe 27 : Noeud MQTT-in 2

🏷️

Name

Avancee bon de preparation

📄 ▼

⚙️

Setup

On Start

On Message

📄

On Stop

```

1  const NB_ESP = 8;
2  const SEUIL_ALERTE = 2; // durée > 2x la moyenne = alerte
3  if (!msg.payload) return null;
4  let payload = msg.payload;
5  if (typeof payload === 'string') payload = JSON.parse(payload);
6  if (payload.status !== 'nouvelle_commande' || !Array.isArray(payload.articles)) return null;
7
8  let compteurs = context.get('compteurParESP') || {};
9  if (compteurs[msg.topic] !== undefined && Object.keys(compteurs).length > NB_ESP) {
10     delete compteurs[msg.topic];
11 }
12 compteurs[msg.topic] = payload.articles.length;
13 context.set('compteurParESP', compteurs);
14
15 if (Object.keys(compteurs).length < NB_ESP) return null;
16
17 let nombreRestant = Object.values(compteurs).reduce((a, b) => a + b, 0);
18 context.set('compteurParESP', {});
19

```

Annexe 28 : Noeud fonction Avancee bon de préparation sur Node-RED

```

const NB_ESP = 8;
const SEUIL_ALERTE = 2; // durée > 2x la moyenne = alerte
if (!msg.payload) return null;

```

Benoist Matys, Stage en IoT
2025-2026

```
let payload = msg.payload;
if (typeof payload === 'string') payload = JSON.parse(payload);
if (payload.status !== 'nouvelle_commande' || !Array.isArray(payload.articles)) return null;

let compteurs = context.get('compteurParESP') || {};
if (compteurs[msg.topic] !== undefined && Object.keys(compteurs).length < NB_ESP) {
  compteurs = {};
}
compteurs[msg.topic] = payload.articles.length;
context.set('compteurParESP', compteurs);

if (Object.keys(compteurs).length < NB_ESP) return null;

let nombreRestant = Object.values(compteurs).reduce((a, b) => a + b, 0);
context.set('compteurParESP', {});

let dernierRestant = context.get('dernierNombreRestantGlobal');
let tempsActuel = Date.now();
let tempsDernierCycle = context.get('tempsDernierCycle');

let nouveauBon = (dernierRestant === undefined || dernierRestant === 0) &&
nombreRestant > 0;
let bonTermine = (dernierRestant !== undefined && dernierRestant > 0 && nombreRestant
=== 0);
let alerte = false;

if (nouveauBon) {
  context.set('totalArticlesBon', nombreRestant);
  context.set('sommeDurees', 0);
  context.set('sommeArticlesPris', 0);
} else if (dernierRestant !== undefined && nombreRestant < dernierRestant &&
tempsDernierCycle) {
  let dureeSecondes = (tempsActuel - tempsDernierCycle) / 1000;
  let articlesPris = dernierRestant - nombreRestant;

  if (dureeSecondes > 0.5 && dureeSecondes < 300) {
    let sommeDureesAvant = context.get('sommeDurees') || 0;
    let sommeArticlesPrisAvant = context.get('sommeArticlesPris') || 0;

    // Moyenne calculée AVANT d'intégrer cette prise, pour comparaison
    let moyenneAvant = sommeArticlesPrisAvant > 0 ? (sommeDureesAvant /
sommeArticlesPrisAvant) : null;
    let dureeParArticle = dureeSecondes / articlesPris;

    if (moyenneAvant !== null && dureeParArticle > moyenneAvant * SEUIL_ALERTE) {
      alerte = true;
    }
  }
}
```

```

    }

    context.set('sommeDurees', sommeDureesAvant + dureeSecondes);
    context.set('sommeArticlesPris', sommeArticlesPrisAvant + articlesPris);
  }
}

context.set('dernierNombreRestantGlobal', nombreRestant);
context.set('tempsDernierCycle', tempsActuel);

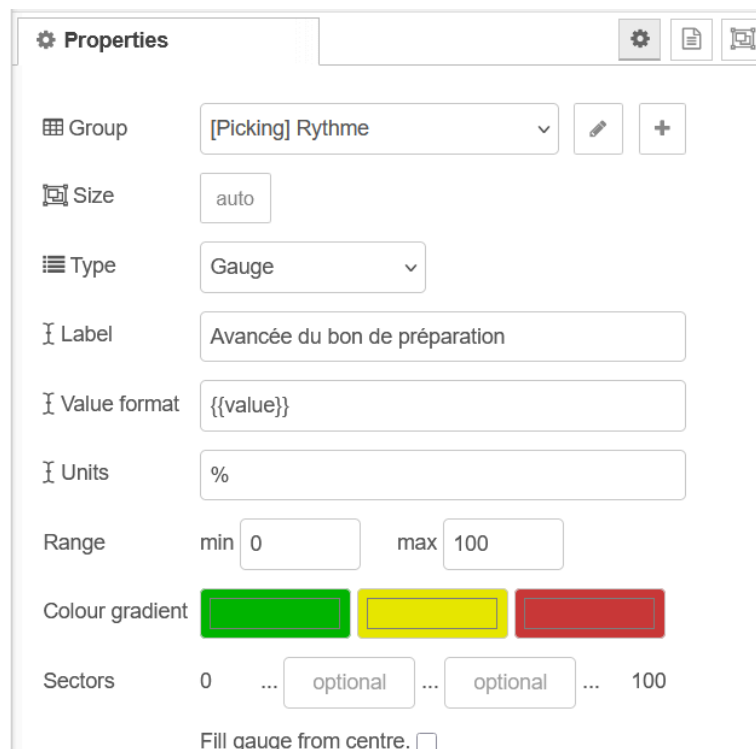
let total = context.get('totalArticlesBon') || nombreRestant;
let pourcentage = total > 0 ? Math.round(((total - nombreRestant) / total) * 100) : 0;

let sommeDurees = context.get('sommeDurees') || 0;
let sommeArticlesPris = context.get('sommeArticlesPris') || 0;
let tempsMoyen = sommeArticlesPris > 0 ? parseFloat((sommeDurees /
sommeArticlesPris).toFixed(1)) : 0;

let tempsRestantSecondes = nombreRestant * tempsMoyen;
let tempsRestantMinutes = parseFloat((tempsRestantSecondes / 60).toFixed(1));
flow.set('tempsDernierCycleGlobal', tempsActuel);
flow.set('sommeDureesGlobal', context.get('sommeDurees'));
flow.set('sommeArticlesPrisGlobal', context.get('sommeArticlesPris'));
let statutPoste;
if (nouveauBon) {
  statutPoste = 'en_cours';
} else if (bonTermine) {
  statutPoste = 'termine';
  flow.set('tempsFinBonGlobal', tempsActuel);
} else if (nombreRestant > 0) {
  statutPoste = 'en_cours';
} else {
  statutPoste = flow.get('statutPoste') || 'attente';
}
flow.set('statutPoste', statutPoste);
return [
  { payload: pourcentage },
  { payload: nombreRestant },
  { payload: tempsMoyen },
  { payload: tempsRestantMinutes },
  { payload: alerte }
];

```

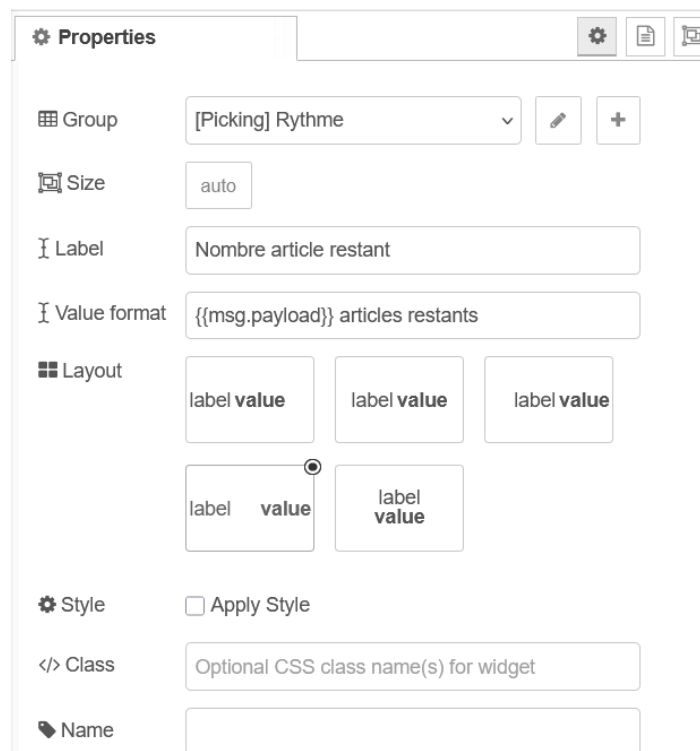
Annexe 29 : Code Avancee bon de preparation sur Node-RED



The image shows the 'Properties' panel for a Gauge widget in Node-RED. The settings are as follows:

- Group:** [Picking] Rythme
- Size:** auto
- Type:** Gauge
- Label:** Avancée du bon de préparation
- Value format:** {{value}}
- Units:** %
- Range:** min 0, max 100
- Colour gradient:** Three color swatches: green, yellow, and red.
- Sectors:** 0 ... optional ... optional ... 100
- Fill gauge from centre:** ☐

Annexe 30 : Noeud de la jauge Avancée du bon de préparation sur Node-RED



The image shows the 'Properties' panel for a Label widget in Node-RED. The settings are as follows:

- Group:** [Picking] Rythme
- Size:** auto
- Label:** Nombre article restant
- Value format:** {{msg.payload}} articles restants
- Layout:** Three 'label value' widgets arranged horizontally, and two 'label value' widgets arranged vertically below them.
- Style:** ☐ Apply Style
- Class:** Optional CSS class name(s) for widget
- Name:** (empty field)

Annexe 31 : Noeud nombre article restant sur Node-RED

⚙️ Properties

⚙️

📄

🖨️

📋 Group

[Picking] Rythme

▼

✎

+

📏 Size

auto

🏷️ Label

Temps moyen par référence

🔢 Value format

{{msg.payload}} s par référence

📱 Layout

label value

label value

label value

label value

label value

⚙️ Style

☐ Apply Style

🔗 Class

Optional CSS class name(s) for widget

🏷️ Name

Annexe 32 : Noeud temps moyen par référence sur Node-RED

⚙️ Properties

⚙️

📄

🖨️

📋 Group

[Picking] Rythme

▼

✎

+

📏 Size

auto

🏷️ Label

Temps restant bon préparation

🔢 Value format

{{msg.payload}} minutes

📱 Layout

label value

label value

label value

label value

label value

⚙️ Style

☐ Apply Style

🔗 Class

Optional CSS class name(s) for widget

🏷️ Name

Annexe 33 : Noeud Temps restant bon préparation sur Node-RED

Annexe 34 : Noeud template 2 sur Node-RED

```
.alerte-led {
  width: 24px;
  height: 24px;
  background: #400;
```

```
border-radius: 50%;  
border: 2px solid #111;  
transition: all 0.2s ease-in-out;  
}  
  
.alerte-led.led-rouge {  
  background: #ff0000;  
  box-shadow: 0 0 15px #ff0000;  
}  
  
.alerte-led.led-vert {  
  background: #15ff00;  
  box-shadow: 0 0 15px #15ff00;  
}  
  
.alerte-label {  
  font-family: sans-serif;  
  font-size: 1rem;  
  font-weight: bold;  
  color: #fff;  
  text-transform: uppercase;  
}  
</style>  
  
<div class="alerte-container">  
  <div class="alerte-led" ng-class="{ 'led-rouge': msg.payload === true, 'led-vert':  
msg.payload === false }"></div>  
  <div class="alerte-label">  
    {{ msg.payload === true ? 'Ralentissement' : 'Rythme normal' }}  
  </div>  
</div>
```

Annexe 35 : Code du template 2 sur Node-RED

Properties

Name: 5 secondes

msg. payload = milliseconds since epoch

msg. topic = a_z

+ add inject now

☐ Inject once after 0.1 seconds, then

Repeat: interval

every 5 seconds

Annexe 36 : Noeud Inject 1 sur Node-RED

Properties

Name: Alerte bloquage

Setup On Start On Message On Stop

```

1 const SEUIL_ALERTE = 2;
2 let tempsDernierCycle = flow.get('tempsDernierCycleGlobal');
3 let sommeDurees = flow.get('sommeDureesGlobal') || 0;
4 let sommeArticlesPris = flow.get('sommeArticlesPrisGlobal') || 0;
5
6 if(!tempsDernierCycle || sommeArticlesPris === 0) return null;
7
8 let moyenne = sommeDurees / sommeArticlesPris;
9 let tempsEcoule = (Date.now() - tempsDernierCycle)/1000;
10 let alerte = tempsEcoule > moyenne * SEUIL_ALERTE;
11 return {payload: alerte};

```

Annexe 37 : Noeud fonction Alerte blocage sur Node-RED

⚙️ Properties

⚙️ 📄 🖨️

Template type

Widget in group ▾

🗪 Group

[Picking] Rythme ▾

✎

+

🖼️ Size

auto

</> Class

Optional CSS class name(s) for widget

🏷️ Name

Name

📄 ▾

📄 Template

↗️

1 <style>

2 .alerte-container {

3 display: flex;

4 align-items: center;

5 justify-content: center;

6 gap: 12px;

7 padding: 12px;

☒ Pass through messages from input.

☒ Add output messages to stored state.

☒ Reload last value on refresh.

Annexe 38 : Noeud du template 3 sur Node-RED

```

<style>
.alerte-container {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 12px;
  padding: 12px;
  background-color: #1a1a1a;
  border: 1px solid #333;
  border-radius: 8px;
}

.alerte-led {
  width: 24px;
  height: 24px;
  background: #400;
  border-radius: 50%;

```

```
border: 2px solid #111;
transition: all 0.2s ease-in-out;
}

.alerte-led.led-rouge {
  background: #ff0000;
  box-shadow: 0 0 15px #ff0000;
}

.alerte-led.led-vert {
  background: #15ff00;
  box-shadow: 0 0 15px #15ff00;
}

.alerte-label {
  font-family: sans-serif;
  font-size: 1rem;
  font-weight: bold;
  color: #fff;
  text-transform: uppercase;
}
</style>

<div class="alerte-container">
  <div class="alerte-led" ng-class="{ 'led-rouge': msg.payload === true, 'led-vert':
msg.payload === false }"></div>
  <div class="alerte-label">
    {{ msg.payload === true ? 'Ralentissement' : 'Rythme normal' }}
  </div>
</div>
```

Annexe 39 : Code du template 3 sur Node-RED

⚙ Properties

🔍 Name1 seconde

≡msg.payload=⌚milliseconds since epoch✕

≡msg.topic=a_z✕

+ add

inject now

☐ Inject once after0.1seconds, then

🔄 Repeatinterval

every1seconds

Annexe 40 : Noeud inject 2 sur Node-RED

Benoist Matys, Stage en IoT
2025-2026

74


Properties






Name




Setup

On Start

On Message


On Stop

```

1  let statut = flow.get('statutPoste') || 'attente';
2
3  // Le statut "termine" reste affiché 5s avant de revenir à "attente"
4  if (statut === 'termine') {
5      let tempsFin = flow.get('tempsFinBonGlobal');
6      if (tempsFin && (Date.now() - tempsFin) > 5000) {
7          statut = 'attente';
8          flow.set('statutPoste', 'attente');
9      }
10 }
11 let chrono = '---:--';
12 if (statut === 'en_cours') {
13     let tempsDernierCycle = flow.get('tempsDernierCycleGlobal');
14     if (tempsDernierCycle) {
15         let secondes = Math.floor((Date.now() - tempsDernierCycle) / 1000);
16         let mm = String(Math.floor(secondes / 60)).padStart(2, '0');
17         let ss = String(secondes % 60).padStart(2, '0');
18         chrono = mm + ':' + ss;
19     }
20 }
21 return [
22     { payload: statut },
23     { payload: chrono }

```

Annexe 41 : Noeud fonction Etat commande et chrono sur Node-RED

Properties

Template type

Widget in group

Group

[Picking] Rythme

Size

auto

</> Class

Optional CSS class name(s) for widget

Name

Name

Template

```
1 <style>
2 .statut-badge {
3     display: flex;
4     align-items: center;
5     justify-content: center;
6     padding: 14px;
7     border-radius: 8px;
8     font-family: sans-serif;
```

Pass through messages from input.

Add output messages to stored state.

Reload last value on refresh.

Annexe 42 : Noeud template 4 sur Node-RED

```
<style>
.statut-badge {
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 14px;
  border-radius: 8px;
  font-family: sans-serif;
  font-size: 1.2rem;
  font-weight: bold;
  text-transform: uppercase;
  color: #fff;
  transition: all 0.3s ease;
}
.statut-badge.attente { background: #333; box-shadow: none; }
.statut-badge.en_cours { background: #1808f3; box-shadow: 0 0 15px #1808f3; }
.statut-badge.termine { background: #15ff00; color: #111; box-shadow: 0 0 15px #15ff00; }
</style>
```

<div class="statut-badg" ng-class="msg.payload">

Benoist Matys, Stage en IoT
2025-2026

```
    {{ msg.payload === 'attente' ? 'En attente' : (msg.payload === 'en_cours' ? 'En cours' :  
    'Bon terminé') }}  
</div>
```

Annexe 43 : Code template 4 sur Node-RED

⚙️ Properties

📁 Group

[Picking] Rythme

⌵

✎

+

📏 Size

auto

🏷️ Label

Chronomètre Picking

🔗 Value format

{{msg.payload}} s

📐 Layout

label value

label value

label value

label value

label value

⚙️ Style

☐ Apply Style

📄 Class

Optional CSS class name(s) for widget

🏷️ Name

Annexe 44 : Noeud chronomètre picking sur Node-RED